Designing Production Agentic Systems
Lecture 2: Agent architectures

Rahul Krishna
Columbia University, COMS E6998-019, Fall 2026

Revised October 16, 2026, as reading for the midterm and the final, from the lecture of September 18, 2026

Abstract

These notes are the reading for Lecture 2, written for students preparing for the midterm and the final. They define the vocabulary of intelligent agents (rational agents, the task environment and its properties) and apply it to `mini-swe-agent`, the course's running example, an agent that patches one reported security weakness at a time. They then take this agent through a fixed workflow, a reactive loop (ReAct), a planner and a validator, and ask of each architecture which decisions we make at design time and which the model makes at run time. The notes close with other architectures from a catalogue of agentic design patterns, and each numbered section ends with a list of key points for revision.

Contents

Course context 1

1 Intelligent agents 2
- 1.1 Rational agents . . . 2
- 1.2 The running example: `mini-swe-agent` . . . 2
- 1.3 A worked example: one SQL injection alert . . . 3
- 1.4 Why this is a rational agent . . . 3
- 1.5 When the agent stops being rational . . . 4
- 1.6 The task environment . . . 5
- 1.7 Properties of the task environment . . . 5
- 1.8 The agent, its tools, and its environment . . . 7
- 1.9 The architectures that follow . . . 8

2 Fixed workflows 9
- 2.1 What a fixed workflow is . . . 9
- 2.2 Who decides, and when . . . 9
- 2.3 Prior knowledge and autonomy . . . 10
- 2.4 What a fixed workflow guarantees . . . 10
- 2.5 Fixed parts and open parts . . . 10
- 2.6 What a fixed workflow gives up . . . 10
- 2.7 Agentless: a published fixed workflow . . . 11
- 2.8 Agentless and SWE-agent . . . 11
- 2.9 Limits of the comparison . . . 12

3 ReAct: reactive agents 12
- 3.1 What a reactive loop is . . . 12
- 3.2 The graph of the loop . . . 13
- 3.3 Conditional edges and compilation . . . 13
- 3.4 Who picks the edge . . . 14
- 3.5 The model's autonomy increases . . . 14
- 3.6 What the loop gains . . . 15
- 3.7 What the loop costs . . . 15
- 3.8 Where the failures occur . . . 15
- 3.9 Shapes of failure inside the loop . . . 16
- 3.10 The two runtime stops . . . 16
- 3.11 An example run . . . 16

4 Planning and validation 17
- 4.1 What the reactive loop cannot answer about itself . . . 17
- 4.2 Open-loop and closed-loop planning . . . 17
- 4.3 The graph of the planner and the validator . . . 18
- 4.4 The edge from `tools` now has two destinations . . . 19
- 4.5 How the runtime decides that the plan has failed . . . 19
- 4.6 What a planner changes compared with a reactive loop . . . 20
- 4.7 What one plan step records . . . 20
- 4.8 Choosing the next action: evidence on early errors . . . 20
- 4.9 Why wider search does not fix myopia . . . 21
- 4.10 Why `submit` can no longer accept a patch . . . 21
- 4.11 Why the validator never reads the conversation . . . 21
- 4.12 Self-review: accuracy and cost . . . 21
- 4.13 Model verifier and sound verifier . . . 22
- 4.14 What the studies imply for a model as verifier . . . 22

5 Other agent architectures 23
- 5.1 Design patterns and who orchestrates them . . . 23
- 5.2 Reason and act . . . 23
- 5.3 Sequential, parallel and loop . . . 24
- 5.4 Loops of generation and evaluation . . . 25
- 5.5 Coordinator and hierarchical task decomposition . . . 25
- 5.6 Swarm, human in the loop, and custom logic . . . 25
- 5.7 Axes for comparing patterns . . . 26
- 5.8 Task ownership and state boundaries . . . 27
- 5.9 Model-call overhead and latency . . . 27
- 5.10 Failure propagation and inspectability . . . 28

Course context

This section places Lecture 2 within the course. It gives the question that motivates the course, the four units of the semester, and what the exams assess. The question also explains why we treat the model as one component of a larger system.

The question that motivates the course

The course is built around the following question:

> How do we place a *probabilistic* model *inside a software system* without surrendering the *guarantees* expected of production software?

In this question, the **model** is a large language model, the component that generates a reply to each request it receives. The table below explains why each phrase in italics belongs in the question.

Phrase	Why it is in the question
Probabilistic	The model samples each reply from a probability distribution, so two calls with the same input can return different outputs. Components that use the model's output must not assume that the outputs agree.
Inside a software system	The model is one component among several. Other components call it, validate its output, and can refuse that output.
Guarantees	A guarantee is a promise that a system makes to its callers: a behavior that a caller is entitled to expect and to rely on every time.

Together, the three phrases describe the problem that the course studies. A production system must keep its guarantees on every call. The model inside it, by contrast, can answer the same request differently each time. The course asks how the other components of the system, which call the model and validate its output, can keep those guarantees.

The agent that we build around the model (Section 1 defines the term) is also one component of a larger system, which we call a

production agentic system. By the end of the semester, we will design such a system.

How the semester is organized

The semester is divided into four units, each devoted to one part of a production agentic system. The table below lists the units in order, with the weeks that each one takes, its subject, and the part of the system that it studies.

Unit (weeks)	Subject	What it studies
1 (1–3)	Agent architectures and control	The orchestrator: who chooses the next operation, and who decides to stop
2 (4–6)	Tool interfaces and concurrency	The tools and the subagents: their contracts, their dependencies, and bounded fan-out (independent operations run in parallel, with a limit on how many run at the same time)
3 (8–9)	Context, state, and persistence	Memory and context: what survives truncation of the context (the input that the model receives in one call), and what survives a restart
4 (10–12)	Deployment, recovery, and observability	The service around the agent, its tools, and its memory: partial failure, retries, and traces

Lecture 2 belongs to Unit 1, agent architectures and control. Unit 1 studies the **orchestrator**, the component that decides which operation runs next and when the agent stops.

What the exams assess

The exams are written and closed book. They assess conceptual understanding, architectural reasoning, and the ability to analyze system behavior.

- **Midterm.** The midterm, on October 23, lasts 40 minutes and covers Units 1 and 2.
- **Final.** The final, on December 17, lasts 40 minutes and covers Units 1 to 4.

Because Lecture 2 belongs to Unit 1, both exams cover the material in these notes.

1 Intelligent agents

Unit 1 of the course studies two questions: who chooses an agent's next operation, and who decides when the agent stops (Course context). These notes answer them for several agent architectures, that is, several ways of organizing the decisions an agent makes. This section supplies the vocabulary needed to compare them: what makes an agent rational, how we specify the task an agent faces, and which property of that task makes it hard. We apply each term to `mini-swe-agent`, an agent that repairs one security weakness in code at a time and serves as the course's running example.

1.1 Rational agents

A rational agent is one that always chooses the action expected to do best by a standard of success that we set for it. To state this precisely, we first define the terms the definition uses.

An **agent** is anything that perceives its **environment**, the part of the world around it, and acts on that environment. What the agent senses at one moment is a **percept**. For a software agent, a percept may be the text it receives after asking to read a file. The agent's **percept sequence** is the complete history of everything it has perceived so far.

An **action** is something the agent does that can change its environment or what it perceives next, such as reading a file or editing it. The **performance measure** is the criterion of success: the standard by which we judge whether what the agent did was good. With these terms, rationality is defined as follows.

> **Definition 1.1** (Rational agent)**.** "For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has." [4, §2.2.2]

The definition asks for the action that is *expected* to do best. It does not ask for the action that turns out best. The expectation is formed from the evidence the agent has. An agent that chooses well on its evidence is therefore rational even when the outcome depends on something it could not perceive.

What is rational at any given time depends on four things:

- **The performance measure.** It defines the criterion of success. Rationality depends on it because an action can be the best one only by some standard. Under a different measure, a different action can be the best one.
- **Prior knowledge.** This is the built-in knowledge of the definition: what the agent knows about its environment before it starts, for example how the code base is laid out. Rationality depends on it because the agent forms its expectation of each action's result from this knowledge together with its percepts.
- **The available actions.** These are the actions the agent can perform. Rationality depends on them because the agent can choose only among them, so the rational action is the best action in this set.
- **The percept sequence to date.** This is everything the agent has perceived up to the moment of choice, and it grows with every new percept. Rationality depends on it because it is the agent's evidence. The same agent can rationally choose differently after it has seen more.

The performance measure, the prior knowledge and the set of actions are all fixed before a run begins. We write down the measure and the set of actions ourselves, and the prior knowledge describes the world in which we place the agent. The percept sequence is the only factor that changes while the agent runs.

We also keep the measure, the prior knowledge and the actions the same across these notes. The architectures they compare therefore differ in one respect: how the agent's next action depends on its percept sequence. The same difference can be stated in terms of time. The architectures differ in which decisions we make at design time, before any run starts, and which are made at run time, after the agent has seen its percepts.

In a fixed workflow, every step and every branch is written before the run. What the agent perceives can change the next step only through a condition that we wrote in advance (Section 2). In a reactive loop, the agent chooses each next action after seeing the latest percept (Section 3).

1.2 The running example: `mini-swe-agent`

The course's running example is an agent called `mini-swe-agent`, which repairs one reported security weakness at a time. Its input comes from a **static-analysis scanner**, a program that reads source code without running it and reports lines that match patterns of known weaknesses. Each report is an **alert**: it names a file, a line and the kind of weakness suspected there.

`mini-swe-agent` works on OWASP BenchmarkPython, a Python web application made up of many small test cases. Each test case is a short program that handles one kind of web request, and some test cases contain deliberate security weaknesses.

The agent reads one alert as an **issue report**, a description of a problem that a developer is asked to fix. It finds the flagged code, edits it, and submits a **patch**, the set of changes it has made to the source files. A **run** is one execution of the agent on one alert. In this design, a run ends when a submission passes a fixed set of checks, described below.

The agent acts through tools. A **tool** is an operation that the agent requests by name, with arguments, and that another program carries out on its behalf. That program is the **runtime**, the code that runs the agent: it executes each tool request, checks each submission, and passes the results back to the agent.

Figure 1 draws the whole agent as a **state machine**, a set of states with transitions between them. Each state is one mode the agent can be in. Each transition is labeled with the event that causes it, and the text in brackets names the information it carries. The machine has three states and five transitions.

- **Controller.** The Controller decides the next action. It either requests a tool, passing the tool's name and arguments (*tool requested*), or asks to submit, passing the patch (*submission requested*).
- **Tool.** The Tool state runs the requested tool and returns its result to the Controller (*tool finished*). The result a tool returns is called an **observation**. An observation is one kind of percept. The agent has three tools, drawn in Figure 1 as a stack of boxes. The tool `read_file` reads a range of lines from a file. The tool `search` finds the places in the code that match a pattern. The tool `edit` replaces one piece of text in a file with another.
- **Submit.** The Submit state validates the submission with a fixed set of checks. If the submission is *accepted*, the run ends. If it is *rejected*, the Submit state returns feedback on the rejection to the Controller, which can continue working.

The Controller's decision can be made by code that we wrote or by a call to the model, the large language model that generates text in reply to the text it is given (Course context). The agent can ask the model to choose the next action.

Which of the two makes the Controller's decision depends on the architecture. Every architecture in the rest of these notes is a variation on this machine: it adds or removes a state or a transition, or changes which component makes a decision.

Later sections draw the machine as a graph, in which each state is a **node** and each transition is an **edge**.

1.3 A worked example: one SQL injection alert

The alert we follow in these notes reports a possible SQL injection. In a **SQL injection**, a value supplied by a user is inserted into the text of a database query, so that a crafted value can change what the query does. The alert comes from Bandit, a static-analysis scanner for Python. It flags a line that builds a query by **interpolation**, that is, by inserting a value directly into the query string (Listing 1).

Listing 1: Before the fix: the value of `bar` is inserted into the query text.

```
46  sql = f'SELECT username from USERS where password = \'{bar}\''
49  cur.execute(sql)
```

The listing omits lines 47 and 48, which open a connection to the database and create `cur`, the cursor object through which line 49 sends the query. The variable `bar` holds a value taken from the incoming request, so the user controls it.

Line 46 copies the characters of `bar` into the query text, and line 49 sends that text to the database, which parses all of it as SQL. A quotation mark inside `bar` is therefore read as part of the query's syntax. If a user sends the value `' OR '1'='1`, the condition becomes `password = '' OR '1'='1'`, which is true for every row, and the query returns every username in the table.

The patch changes two lines (Listing 2). Line 46 now holds a **placeholder**, the question mark, where the value used to be. Line 49 passes `bar` to the database as a separate argument, called a **bound parameter**, rather than as part of the query text.

Listing 2: After the fix: the query text holds a placeholder, and the value is passed separately as a bound parameter.

```
46  sql = 'SELECT username from USERS where password = ?'
49  cur.execute(sql, (bar,))
```

The fix works because the database now receives the query text and the value separately. It first parses the text, which contains no user input. Parsing settles the structure of the query before any value is attached: which table is read, which column is compared and what condition selects the rows. Only then does the database attach the value to the placeholder, as data to compare against the password column. The characters of the value, quotation marks included, are never parsed as SQL, so the value can no longer change the structure of the query.

Figure 2 shows an example run of `mini-swe-agent` on this alert, read from top to bottom. The arrows that point to the Controller are percepts, and the arrows that point away from it are actions. The first percept is the alert. The agent then reads the region around the flagged line, and the result shows that line 46 interpolates `bar` and that line 49 executes the query. The agent edits those two lines and submits a patch of two lines in one file.

At any point in the run, the agent's percept sequence consists of the arrows toward the Controller above that point. It holds first the alert alone, then the alert and the excerpt that the read returned, and so on. The edit also returns a confirmation, which is a percept, but the figure does not draw it.

The Submit state accepts the patch because the patch passes each of its checks, and the run then ends.

- **The patch applies.** Its changes can be made to the current files without conflict.
- **The patch changes the flagged line.** Here, the flagged line is line 46.
- **The rescan no longer reports the alert.** A **rescan** is a second run of the scanner on the patched code. A rescan that no longer reports the alert is called **clean**.

1.4 Why this is a rational agent

On the worked alert, `mini-swe-agent` acts rationally when it reads the flagged code before editing it. To check this claim, we first state, for our agent, the four things on which rationality depends.

- **The performance measure.** The agent earns one point if its patch removes the reported weakness and leaves the endpoint behaving as it did. The run must also stay within a limit on its cost. An **endpoint** is a part of the application that answers one kind of request. Here, it is the endpoint that contains the flagged code.
- **Prior knowledge.** The agent knows the application's layout and what each tool returns.
- **The available actions.** The agent can call `read_file`, `search` and `edit`, and it can `submit` a patch.
- **The percept sequence to date.** It is made up of the alert, whatever the tools return, and the result of each submission. The agent perceives nothing else.

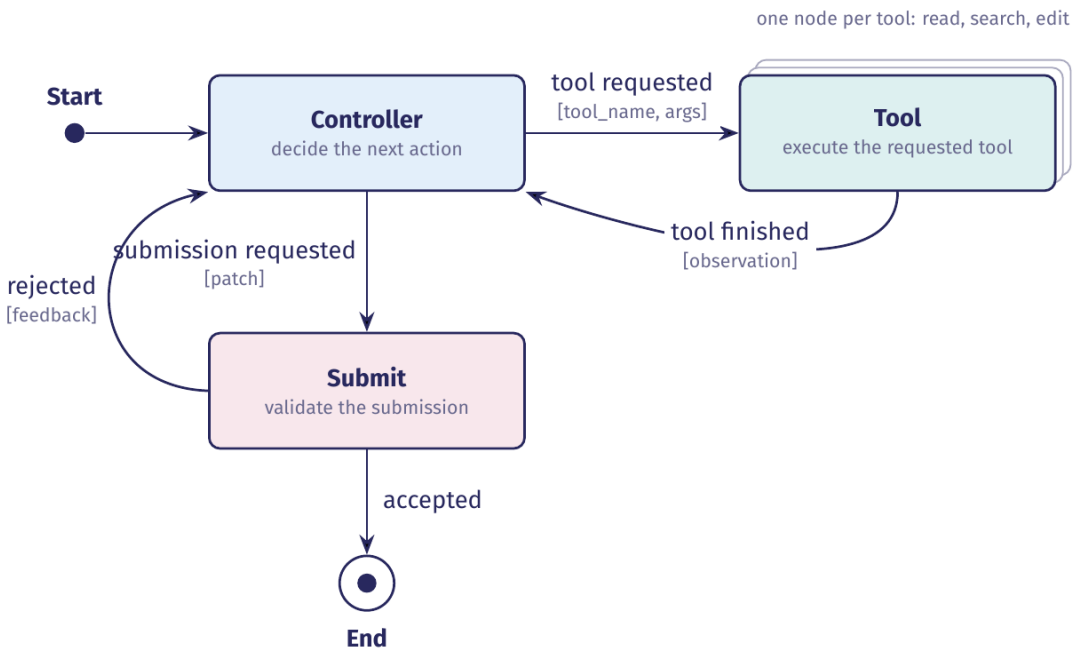


Figure 1: `mini-swe-agent` as a state machine with three states (Controller, Tool and Submit) and five transitions. Each transition is labeled with the event that causes it, followed in brackets by the information it carries.

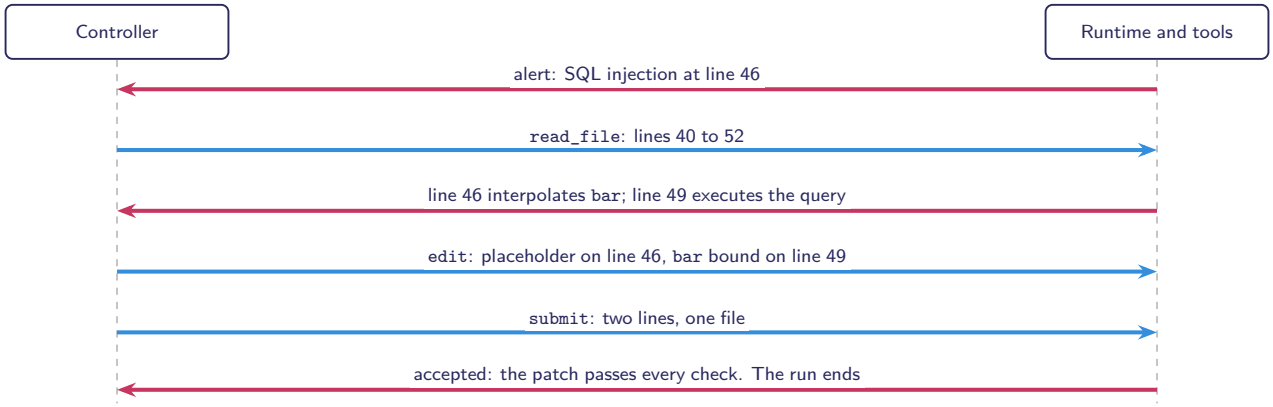


Figure 2: An example run of `mini-swe-agent` on the worked alert, as messages between the Controller and the runtime. Arrows toward the Controller carry percepts, and arrows away from it carry actions.

Given these four, the rational first action is to read the flagged lines before editing them. When the agent makes its first choice, its percept sequence holds only the alert. The alert names a file, a line and a kind of weakness, but it does not show the code. An `edit` names the exact text it replaces, so an edit written without seeing the code could fail to apply, change the wrong text, or alter what the endpoint returns.

A read costs one action and returns the code that the patch must change, including the exact text that an edit has to name. Reading first is therefore the action expected to maximize the performance measure. The read does not guarantee a correct patch. It makes one more likely, and the definition judges the agent by its expected performance rather than by the outcome.

After the read, the percept sequence also holds the excerpt, and the rational choice changes. The agent now knows which line builds the query and which line executes it, so editing those lines becomes the action expected to do best. The same program chooses differently because its percept sequence has grown.

The judgment that `mini-swe-agent` is rational holds only relative to the four things listed above. If the measure, the prior knowledge, the actions or the percepts change, a different action may become the rational one. Rationality is therefore a property of a program together with these four, and not of the program alone.

Stating the four also makes the claim that an agent is rational checkable. We can ask whether any other available action was expected to do better under the same measure, knowledge and evidence.

1.5 When the agent stops being rational

Behavior that is rational under one performance measure, or in one environment, can stop being rational when either changes. Some behavior is not rational under our measure at all. In each of the following cases, the behavior described does not maximize expected performance, and a better agent behaves differently.

- **Optimizing per tool call.** Suppose the measure also charges for each tool call, and a call costs more the more text it returns. An agent that reads whole files then fares poorly, because it

pays for text it does not need. A better agent reads the smallest range of lines that answers its question.
- **Patching only the flagged line.** The weakness is the path by which an untrusted value reaches the query. The flagged line is only the place where the scanner recognized one part of that path. The value may reach the query through a helper function that the alert never names. A patch to the flagged line alone can then leave part of that path in place, and the weakness survives. A better agent first follows the value back to where it comes from.
- **Editing after acceptance.** In Figure 1 the run ends at acceptance, so this case arises only in a design that leaves the decision to stop to the agent. An agent that keeps editing after its submission has been accepted gains nothing under the measure, and every further edit risks breaking the endpoint's behavior. A better agent stops.

Each case becomes an explicit design decision in the architectures that follow:

- **Cost.** In the reactive loop, the runtime enforces a **budget**, a limit on the number of model calls a run may make. Once the budget is spent, the runtime refuses to issue another model call (Section 3.10). Better tools also let the agent request only the text it needs. Lecture 4 studies tools and their contracts.
- **Depth of investigation.** In the planner, the plan sets how deep the investigation goes (Section 4).
- **Termination.** The decision to end a run becomes a stopping rule that we write and the runtime applies, such as the budget and the stall check of the reactive loop (Section 3.10). Lecture 3 returns to these rules and asks when they guarantee that a run ends.

A separate gap lies in the checks rather than in the agent. The Submit state checks less than the performance measure asks. It checks that the patch applies, that it changes the flagged line, and that the rescan no longer reports the alert. The measure asks that the weakness be gone and that the endpoint still behave as it did.

No check runs the endpoint. A rescan that reports nothing shows only that the scanner no longer recognizes the pattern it flagged. An accepted submission is therefore not necessarily a correct one. A patch to the flagged line alone, when the unsafe value comes from a helper function, can pass every check while the weakness survives.

1.6 The task environment

Before designing an agent, we write down the task it must perform. That specification is the **task environment**. It has four parts, a performance measure (P), an environment (E), actuators (A) and sensors (S), and is written down as a **PEAS description** [4, §2.3.1].

- **Performance measure.** It states what counts as success.
- **Environment.** It is the world in which the agent acts.
- **Actuators.** They are the means by which the agent acts on its environment. For a software agent, they are the operations it can request, such as its tools. An action is one use of an actuator, for example one call to `read_file` with a particular path and range.
- **Sensors.** They are the means by which the agent perceives its environment. For a software agent, they are the channels through which inputs and results are handed back to it. A percept is what arrives through a sensor at one moment, for example the lines that one read returned.

In our task environment, the code base is taken at a **pinned commit**, one fixed version of its history, so that every run starts from the same files. The **working tree** is the copy of those files that the agent reads and edits during a run. Among the sensors, the **validation result** is the Submit state's verdict on a submission: accepted, or rejected with feedback.

Table 1 fills in the four parts for `mini-swe-agent`. In the table, the performance measure asks for a correct patch at bounded cost, that is, within a limit on what the run may spend. The actuators are exactly the agent's tools and its submission.

Table 1: PEAS description of the task environment for `mini-swe-agent`.

	For `mini-swe-agent`
Performance measure	The reported weakness is gone, and the endpoint still returns what it returned before, at bounded cost
Environment	OWASP BenchmarkPython at a pinned commit, one scanner alert read as an issue report, and the working tree the agent edits
Actuators	`read_file(path, range)`, `search(pattern)`, `edit(path, old, new)`, `submit(patch)`
Sensors	The alert, file excerpts, search hits, edit confirmations, the validation result, and tool errors

We specify the task environment as fully as possible before designing the agent, because the right design depends on it. A property of the task, such as how much of it the agent can see, determines what the agent must do, for example whether it must act to gather information (Section 1.7). The architectures in the rest of these notes all keep the task environment of Table 1 fixed and differ only in how the agent chooses its actions.

1.7 Properties of the task environment

A task environment can be classified along seven dimensions, each a pair of opposite properties [4, §2.3.2]. Where a task sits on each dimension tells us what the agent must cope with. Each table below gives the definitions of the two properties and places familiar tasks on each side. The tasks include solving a crossword, playing chess or poker, driving a taxi and diagnosing a patient.

Several of the definitions refer to the state of the environment. The **state** of an environment is everything about it at one moment that can matter to the task, for example the current contents of every file. This is a different use of the word from the states of Figure 1, which are modes of the agent.

On each dimension, one side makes an agent easier to design: fully observable, single agent, deterministic, episodic, static, discrete and known. Our task environment is on the easier side of every dimension except observability and the episodic or sequential dimension. It is partially observable, and it is sequential within one alert but episodic across alerts. Partial observability is the property that makes the task hard, as the first dimension below explains.

Fully or partially observable.

This dimension asks whether the agent's sensors show it everything it needs in order to choose an action. An environment is **fully observable** when they do, and **partially observable** when some relevant part of the state is hidden or reported inaccurately. A crossword is fully observable, because the whole grid is in front of the solver. Poker is partially observable, because the other players' cards are hidden.

	Fully observable	**Partially observable**
Definition	The sensors give the agent the complete state of the environment at each point in time, or at least every part that matters for the choice of action.	The sensors are noisy or inaccurate, or part of the state is simply missing. With no sensors at all, the environment is unobservable.
Examples	Crossword, chess with a clock, backgammon, image analysis	Poker, taxi driving, medical diagnosis, part-picking robot, refinery controller

Our task environment is partially observable. The agent sees only the parts of the code that it reads. It never sees whether the endpoint still works, because none of its actions runs the application. Nor does it see the setting in which the application is deployed: the machine that runs it, the operating system, or the other programs running beside it.

This property is what makes the task hard. The evidence a correct patch depends on, such as where an unsafe value comes from, reaches the agent only through actions it chooses to take. In a partially observable environment, a rational agent must therefore act in order to gather information, as `mini-swe-agent` does when it reads before it edits.

Single agent or multiagent.

This dimension asks whether any other entity in the environment is pursuing a goal that depends on what our agent does. The environment is **single agent** when no such entity exists, and **multiagent** when one does. A crossword is single agent. Chess is multiagent and competitive, because the opponent's success depends on our moves. Traffic is multiagent and partly cooperative, because drivers try to avoid colliding with one another.

	Single agent	**Multiagent**
Definition	No other entity behaves as though it were maximizing a measure that depends on what this agent does.	Another entity is best described as maximizing a measure whose value depends on this agent's behavior. It can compete, as in chess, or cooperate, as in traffic.
Examples	Crossword, medical diagnosis, image analysis, part-picking robot, refinery controller	Chess with a clock, poker, backgammon, taxi driving, English tutor

Our task environment is single agent. The working tree, the tools and the Submit checks respond to the agent's actions, but none of them pursues a goal of its own. Nothing the agent interacts with is trying to cooperate with it or to beat it.

Deterministic or nondeterministic.

This dimension asks whether the current state and the agent's action together settle the result of the action. In a **deterministic** environment, the current state and the agent's action completely determine the next state. In a **nondeterministic** environment they do not, so the same action in the same state can lead to different results. Filling in a square of a crossword has exactly one result. A move in backgammon depends on a roll of the dice.

By convention, uncertainty that comes only from another agent's choices does not count, which is why chess appears in the deterministic column. A **stochastic** environment is a nondeterministic one in which the possible results are given probabilities. A partially observable environment can look nondeterministic even when it is not, because the part of the state that decides the result is hidden from the agent.

	Deterministic	**Nondeterministic**
Definition	The current state and the action the agent executes fix the next state completely.	They do not. A partially observable environment can look nondeterministic even when it is not. Stochastic is the narrower word, for models that put numbers on the possibilities.
Examples	Crossword, chess with a clock, image analysis	Poker, backgammon, taxi driving, medical diagnosis, refinery controller

Our task environment is deterministic: the same read, or the same rescan, on the same files returns the same result every time. Determinism is a property of the environment, and it does not mean that every run of the agent takes the same path. The model's replies are *sampled*: the model draws each reply at random from a range of likely replies, so the same request can produce different replies. An edit also changes the files that later reads return. Two runs from the same state can therefore still take different paths, as the example run of the reactive loop shows (Section 3.11).

Episodic or sequential.

This dimension asks whether the agent's current decision affects its later ones. In an **episodic** environment, experience divides into separate episodes. In each one the agent perceives and then acts once, and no episode depends on the actions taken in earlier ones. In a **sequential** environment, the current decision can affect every decision that follows. A part-picking robot that sorts parts from a conveyor belt works episodically, because each part is judged on its own. Chess is sequential, because each move constrains the moves that follow.

	Episodic	**Sequential**
Definition	Experience divides into separate episodes. The agent perceives, then acts once, and the next episode owes nothing to the last.	The current decision can affect every decision that follows.
Examples	Image analysis, part-picking robot	Crossword, chess, poker, backgammon, taxi driving, medical diagnosis, refinery controller, English tutor

Our task environment is both, at different scales. It is sequential within one alert, because what the agent reads next depends on what it has just read, and an edit changes what later reads return. It is episodic across alerts if we treat the whole run on one alert as a single episode. No run depends on an earlier one, because the agent carries nothing from one alert to the next.

Static or dynamic.

This dimension asks whether the environment can change while the agent is deciding what to do. A **static** environment cannot, so the time the agent spends deciding costs it nothing. A **dynamic** environment can, as traffic does while a taxi driver hesitates. An environment is **semidynamic** when the environment itself does not change with time but the agent's score does, as in chess played with a clock.

	Static	Dynamic
Definition	The environment cannot change while the agent is thinking, so time spent deciding costs nothing.	The environment can change while the agent is thinking. Semidynamic if the environment does not change with time but the score does.
Examples	Crossword, poker, backgammon; chess with a clock and image analysis are semidynamic	Taxi driving, medical diagnosis, part-picking robot, refinery controller, English tutor

Our task environment is static. The working tree changes only when the agent edits it, so nothing changes while the agent is deciding what to do next.

Discrete or continuous.

This dimension asks whether the state, time, percepts and actions take a finite number of distinct values. In a **discrete** environment they do, as the positions and moves of chess do. In a **continuous** environment they range smoothly over values, as the speed and position of a taxi do.

	Discrete	Continuous
Definition	The state, the handling of time, and the agent's percepts and actions all come in a finite number of distinct values.	State, time, percepts or actions sweep through a range of values.
Examples	Crossword, chess with a clock, poker, backgammon, English tutor	Taxi driving, medical diagnosis, image analysis, part-picking robot, refinery controller

Our task environment is discrete. Files, lines, edits and **hunks**, the blocks of changed lines that make up a patch, can all be counted. Nothing in the task is a quantity measured on a continuous scale.

Known or unknown.

This dimension concerns what is known about how the environment works, rather than the environment itself. An environment is **known** when the agent, or its designer, knows the rules that govern it: what result each action produces in each state, or with what probabilities. It is **unknown** when the agent must first learn those rules before it can choose well.

Knowing the rules does not mean knowing the current state. Whether the agent can see the current state is the separate question of observability. A new video game shows the whole screen, so it is fully observable. It does not say what the buttons do, so it is unknown.

	Known	Unknown
Definition	The outcomes of every action are given. This is about what the agent knows, not about the environment itself.	The agent has to learn how the world works before it can choose well. A new video game shows the whole screen and still will not say what the buttons do.
Examples	Chess and poker: the rules can be handed over in full	Driving a rented car in a country you have not driven in

Our task environment is known, because we know what each tool does. What the agent does not know is where the weakness is and how the unsafe value reaches it. That is a matter of observability rather than of knowing how the world works: the information is in the code, but the agent sees it only by reading.

1.8 The agent, its tools, and its environment

The terms of Table 2 each name a part of `mini-swe-agent` or of its environment. Figure 3 shows these parts by placing the state machine of Figure 1 inside its environment. Each label in magenta names a term that Table 2 defines for our agent. The figure adds the following to Figure 1.

- **The environment.** The dashed outer box is the environment: the working tree at the pinned commit, and the alert.
- **The working state.** The Controller reads and updates a **working state**, the agent's own record of what the run has found so far: code locations, excerpts, edits and cost. Figure 1 did not draw it.
- **One box per tool.** The Tool state is split into one box per tool, reached by dotted lines, because the Controller requests a tool by its name.

The solid box is the **agent program**, the code that chooses the agent's actions from its percepts. It contains the Controller, the working state and the Submit state.

The agent program runs on machinery that executes it and carries out its actions. This machinery consists of the operating-system process that runs the program, the **model client** (the code that sends requests to the model), and the code behind each tool. In the vocabulary of Table 2, this machinery is called the **architecture**. Everywhere else in these notes, *architecture* has a different meaning: a way of organizing the agent program, such as a fixed workflow or a reactive loop.

The figure also shows how the agent program reaches its environment and how a run ends.

- **Access to the environment.** Apart from the Submit checks, which run the scanner on the working tree, the tools are the only way the agent program reaches the environment. Every percept after the alert, other than the verdict on a submission,

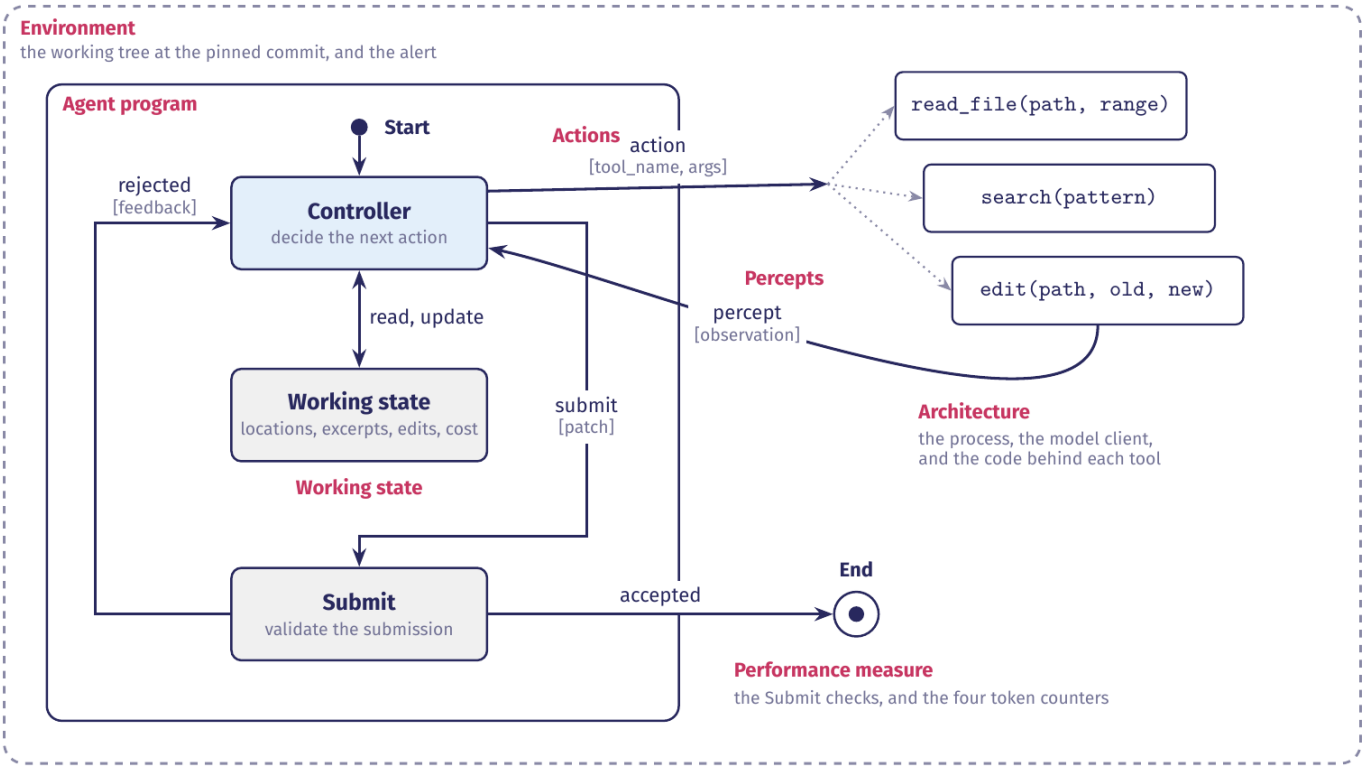


Figure 3: `mini-swe-agent` inside its environment: the state machine of Figure 1, with the working state drawn and one box per tool. Each label in magenta names a row of Table 2.

comes back from a tool, and every change to the working tree is made by the `edit` tool.
- **Ending a run.** The only transition to the end leaves the Submit state, so in this machine a run ends only when a submission is accepted. Later architectures add other ways to stop a run, such as a budget (Section 3.10).

The size of a model's input and output is measured in **tokens**, small pieces of text that are often parts of words. Four counters record how many tokens the run has used, one for each kind of token that the model's provider reports, such as input and output tokens. The counters track the cost that the measure bounds. Lecture 3 shows how the runtime adds the tokens of each model call to them.

The performance measure is not part of the agent program. It is our standard for judging a run from outside the program. Inside the program, the Submit checks and the token counters stand in for parts of it. The Submit checks test part of what the measure asks, though less than all of it (Section 1.5).

Table 2 lists each term with the part of `mini-swe-agent` it names.

1.9 The architectures that follow

The rest of these notes takes `mini-swe-agent` through a sequence of architectures. The fixed workflow, the reactive loop and the planner let the agent decide progressively more of its run for itself. The validator then adds a separate model call that judges a patch only after the Submit checks have passed it.

- **Fixed workflow.** We write every step and its order before the run, and the model only supplies content at the points we choose (Section 2).
- **Reactive loop (ReAct, short for reasoning and acting).** The model chooses the next action after seeing each result. Its graph has the edge from the tools back to the Controller that Figure 1 draws, and the fixed workflow's graph lacks it (Section 3).
- **Planner.** The agent writes its intended sequence of steps into its working state, where the plan can be read and revised as the run proceeds (Section 4).
- **Validator.** A new node, the validator, judges each patch that has passed the Submit checks, and it becomes the one component that can end a run with an accepted patch (Section 4.10).
- **Other agent architectures.** The notes close with a survey of further architectures from a catalogue of agentic design

Table 2: The vocabulary of intelligent agents, with the part of `mini-swe-agent` that each term names. Each row matches a label in magenta in Figure 3.

Term	In `mini-swe-agent`
Environment	The working tree at the pinned commit, and the alert
Agent program	The Controller, which decides the next action; the working state it reads and updates; and the Submit state, which validates the submission
Working state	Locations, excerpts, edits, cost
Actions	`read_file(path, range)`, `search(pattern)` and `edit(path, old, new)`, each requested by name with its arguments, and `submit` with a patch
Percepts	The alert, file excerpts, search hits, edit confirmations, the validation result, and tool errors
Architecture	The process, the model client, and the code behind each tool
Performance measure	The Submit checks, and the four token counters

patterns (Section 5).

Across these architectures, the agent's task, the alert and the tools stay the same, and only the architecture changes. This is what makes the comparison worth making. A difference between two architectures on the same alert cannot then be explained by a change in the task or the tools.

Key points

- A rational agent selects, for each percept sequence, the action expected to maximize its performance measure, given the evidence of its percepts and its built-in knowledge.
- Rationality depends on the performance measure, prior knowledge, the available actions and the percept sequence to date, so it is a property of a program relative to these four rather than of the program alone.
- Reading the flagged code before editing it is the rational first action for `mini-swe-agent`, because the environment is partially observable and the code reaches the agent only through its own reads.
- The Submit checks test less than the performance measure asks, so an accepted patch is not necessarily a correct one.
- A task environment is specified by its performance measure, environment, actuators and sensors (PEAS), and specifying it fully comes first because the right design depends on it.
- Our task environment is partially observable, single agent, deterministic, static, discrete and known, and it is sequential within one alert and episodic across alerts.
- The architectures in these notes keep the task environment fixed and differ only in how the agent's next action depends on its percept sequence.

2 Fixed workflows

The architectures in these notes differ in which decisions about a run the designer makes before it starts and which are made while it runs (Section 1). The first one we study, the fixed workflow, leaves every decision about how a run proceeds to the designer, who makes them before the program runs. In exchange, the program gains guarantees that the later architectures give up, which makes the fixed workflow the baseline for comparing them. We explain the idea with `mini-swe-agent`, the course's running example, and then examine Agentless, a published system built the same way.

2.1 What a fixed workflow is

A **fixed workflow** is a program whose steps, and their order, are decided by the designer before any task arrives. A **step**, also called an operation, is one unit of work the program performs, such as reading lines of a file, calling the model once, or submitting a patch. The model is still called, but only to produce content at points the designer has chosen, for example to rank candidate files or to write a patch. It never decides what the program does next.

A run is one execution of the program on one task (Section 1.2), which in our running example is one scanner alert. It helps to separate two kinds of decisions about a run. **Control decisions** determine how the run proceeds. **Content decisions** determine what each step produces, such as which file to patch and what the patch says. The control decisions cover four things:

- **Operations.** The designer decides which step runs next, and whether any step follows it.
- **Model calls.** The designer decides how many times the run calls the model.
- **Termination.** The designer decides what ends the run.
- **Artifacts.** The designer decides which artifacts each step reads and writes. An **artifact** is a named output of one step that a later step can read, such as a list of candidate lines or a patch.

A decision is made at **design time** when we, as the designer, make it while writing the program, before any task arrives. It is made at **run time** when the program, or the model it calls, makes it during a run. In a fixed workflow, we make every control decision at design time, and the model makes content decisions at run time. No output of the model can change a control decision.

In our running example, `mini-swe-agent` receives one scanner alert that flags a line of code, and its task is to patch that code. A fixed workflow for this task could consist of four steps: read the region around the flagged line, ask the model to name the lines to edit, ask the model to write the patch, and submit the patch to a fixed set of checks that accept or reject it. Every alert goes through the same steps, once each, in the same order. We refer to this fixed-workflow version of `mini-swe-agent` as **the chain**, because its steps run one after another in a sequence set in advance.

2.2 Who decides, and when

Every decision in a fixed workflow has an owner and a time at which it is made (Table 3). In the table, "Us" means the designer. The rows above the heavy rule are control decisions, which we make at design time. The rows below it are content decisions, which the model makes at run time.

Table 3: Decisions in a fixed workflow, with who makes each one and when. The heavy rule separates the control decisions, made by the designer, from the content decisions, made by the model.

Decision	Who decides	When
Which operation runs next	Us	Design time
Whether anything runs after it	Us	Design time
How many model calls the run makes	Us	Design time
What ends the run	Us	Design time
What a model call returns	The model	Run time
Which values travel to the next step	The model	Run time

Both of the model's decisions concern values. The model decides what a call returns, for example the text of a patch. It also decides which values travel to the next step, for example which candidate files it ranks highest and therefore which files the next step receives. This does not conflict with the designer's control over artifacts. The designer decides that the next step reads the artifact holding the ranked files. The model decides which files that artifact contains.

This division is about authority, not difficulty. Choosing the right file to patch is often the hardest part of the task, and it is left to the model. The designer keeps a comparatively easy decision, the order of the steps, for example that a file is chosen before its lines are read. That decision carries the authority over what happens next.

A fixed workflow need not be a straight line of steps. It may branch, filter a set of candidates, or repeat a step a set number of times. A **predicate** is the condition in the code that selects a branch. The workflow remains fixed because we wrote every predicate it tests and we set every number of repetitions.

Which branch a particular run takes can depend on data produced during the run, such as which candidate patches pass the tests. The set of possible paths cannot, so we can list every path before the program runs. Agentless (Section 2.7) is an example: it samples a fixed number of candidate patches and filters them by running tests.

The model's output is treated as data. A step may store it, pass it to the next step, or write it into a file. The program never reads it as an instruction that names the next step. This is why no output of the model can change a control decision.

2.3 Prior knowledge and autonomy

A fixed workflow knows about its task only what its designer put into it. Every control decision records an expectation we formed before any particular task was seen. In the vocabulary of Section 1.1, these decisions rely on prior knowledge, the knowledge built in by the designer. They do not rely on the program's percepts, the information it receives during the run, such as the contents of the files it reads.

> **Definition 2.1** (Autonomy)**.** "To the extent that an agent relies on the prior knowledge of its designer rather than on its own percepts and learning processes, we say that the agent lacks autonomy." [4, §2.2.3]

In its control decisions, a fixed workflow lacks autonomy in exactly this sense. What it reads during a run can change the content of a step, for example the patch the model writes. It can change which step comes next only through a condition that we wrote in advance, so the choice of the next step still rests on our prior knowledge.

Relying on prior knowledge is a legitimate engineering choice when that knowledge is correct, that is, when we know in advance which steps the task needs. Writing those steps down then gives the guarantees of shape, address and termination (Section 2.4). The architectures in the following sections rely less on the designer's prior knowledge, one decision at a time, starting with the choice of the next action (Section 3).

2.4 What a fixed workflow guarantees

Fixing the control decisions gives three guarantees, called shape, address and termination. Each one follows from the fact that the designer, and not the model, decides how the run proceeds.

- **Shape.** The number and order of model calls are known before the run, because they are the calls we wrote. The number of calls does not depend on what the model says. We can therefore estimate the cost of a task before its run starts, and the cost of a thousand alerts before running any of them. In the chain, every alert makes the same two model calls, whatever the alert says.
- **Address.** A wrong result can be traced to the step that produced it, because each artifact is written by one step that we know in advance. The name reflects this: every failure has a location in the program. Debugging a bad patch in the chain means reading its four artifacts in order: the excerpt it read, the lines the model named, the patch the model wrote and the results of the checks. It does not require reconstructing a long sequence of choices made by the model, such as the sequence of up to forty model calls that a reactive loop can make (Section 3.10).
- **Termination.** The run ends when its last step returns. Stopping does not depend on anything the model says, because the last step is fixed before the run starts. The chain ends when its submit step returns.

Each guarantee is a property of the program we have designed. None of them is a prediction about what the model will do. The shape guarantee, for example, tells us how many calls a run makes. It says nothing about whether the resulting patch is correct.

Termination is the guarantee that the later architectures never regain. A reactive loop, a planner and a validator all let the model choose further steps, so the program no longer has a last step that is known before the run. These architectures replace termination with a budget, the limit on the number of model calls a run may make (Section 1.5). A budget guarantees that the run stops. It does not guarantee that the run finishes, that is, reaches an accepted submission before the budget is spent.

In more autonomous agents, these guarantees no longer follow from the structure of the program. Where we need one, we have to add a mechanism that provides it, as a budget on model calls provides stopping.

2.5 Fixed parts and open parts

Most systems we build are partly fixed and partly open, and where the boundary falls is a design decision. We divide a system into **stages**, each a group of steps that produces a defined result for the next stage. A **fixed stage** makes all its control decisions at design time, as a fixed workflow does. An **open stage** lets the model decide what runs next, as the architectures of the following sections do.

The course's first homework, HW1, shows this division. It asks for a triage application for scanner alerts. The application's intake stage, which produces the alerts the application will investigate, is a fixed workflow. Its investigation loop, in which the model chooses the next step, is an open stage.

Knowing which stages are fixed matters because budgeting, recovery, transactions and parallel execution are all easier to engineer in a fixed stage:

- **Budget.** We can estimate the cost of a fixed stage before it runs, because we chose how many model calls it makes. For an open stage we can only reserve an amount and enforce a limit, because the number of calls depends on the model's choices.
- **Recover.** A **checkpoint** saves a run's progress so that a failed run can resume from that point instead of starting over. A checkpoint needs named artifacts to save and to resume from. A fixed stage names its artifacts before the run, and its boundary with the next stage is where it hands on its defined result, so that boundary is a natural checkpoint.
- **Transact.** A **transaction** groups a set of changes so that either all of them take effect (a commit) or none of them do (a rollback). To decide where to commit and what to roll back, we must know the sequence of steps in advance.
- **Run in parallel.** Two stages can safely run at the same time only if neither one writes what the other reads or writes. A fixed stage declares in advance which artifacts it reads and writes, so this can be checked before the run.

In an open stage each of these is harder, because the steps and their artifacts are not known until the run happens. The architectures in the rest of these notes pay this cost.

2.6 What a fixed workflow gives up

A fixed workflow pays for its guarantees with its ability to respond to what a run discovers. Because every step and every branch was written before the task arrived, the program can respond to new evidence only in the ways the designer foresaw. It cannot take a step the designer did not write. In particular, it cannot look again, notice what is missing, or ask for more:

- **No second look.** Every read is scheduled before we know what the first read returns, so no read can be chosen in response to an earlier one.
- **No sense of what is missing.** Nothing in the run represents evidence the program has not read, so the program cannot tell that its information is incomplete.
- **No way to ask.** If the fix depends on a file the alert never names, no step exists in which the program could go and read it.

A fixed workflow chooses its steps from prior knowledge alone, so it is rational as long as that knowledge is correct, that is, as long as the steps we wrote are the ones the task needs. When the knowledge is wrong, the workflow stops being rational. Here *rational* has the meaning given in Section 1.1: a rational agent chooses the action expected to maximize its performance measure, given what it has perceived and what it was built to know.

Our task environment is partially observable (Section 1.7), which

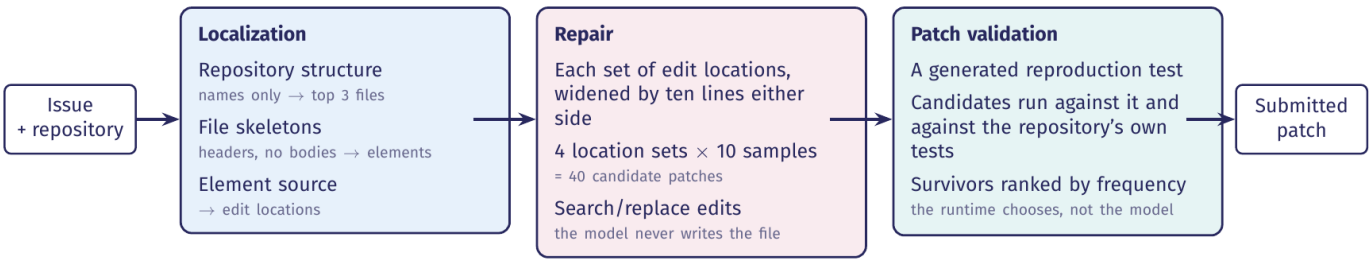


Figure 4: The three phases of Agentless, run in a fixed order from an issue and its repository to a submitted patch. The model supplies content within each phase, but no output of the model can change the order of the phases. Redrawn after Figure 1 of Xia et al., FSE 2025.

means that the agent sees only the parts of the code it reads. In such an environment, a rational agent must act in order to gather information, because the evidence it needs reaches it only through its own actions. A fixed workflow does gather information, because its steps read files. It reads them, however, on a schedule we wrote before we knew what the reads would return.

When our prior knowledge is incomplete, a fixed workflow fails silently. A **silent failure** is one that the system neither detects nor reports, so the run ends as if it had succeeded.

Consider an alert about a SQL query built from untrusted input, in which the untrusted value reaches the flagged line through a helper function in a file the alert never names. The chain reads only the region around the flagged line, so it never sees the helper. It does not stop early or raise an error, and it does not report a gap in its evidence. It writes a patch for the code it was given and submits it, and the checks run at submission pass.

The checks pass because they test less than the performance measure asks (Section 1.5). They confirm that the patch applies, that it touches the flagged line, and that the rescan is clean. A clean rescan shows that the scanner no longer recognizes the flagged pattern. It does not show that the untrusted value can no longer reach the query.

Nothing in the chain records what it did not read, so nothing in the run signals a problem. A fixed workflow gathers only the evidence its designer scheduled, so a gap the designer did not foresee stays invisible to it.

The next architecture, the reactive loop, adds the ability to look again. In a reactive loop, the model sees each tool result and then chooses the next action, so it can decide to read the helper (Section 3).

2.7 Agentless: a published fixed workflow

Agentless is a published system for repairing software issues, and it is a fixed workflow. It was evaluated on **SWE-bench Lite**, a benchmark of 300 issues from open-source Python repositories. For each issue, a system receives the issue description and the repository and must produce a patch. The issue counts as **resolved** when the patch passes the benchmark's tests for that issue. In its published comparison, Agentless resolved more of these issues than any open-source agent it was compared with [8].

Agentless runs three phases in a fixed order (Figure 4). Each phase is a stage in the sense of Section 2.5, handing a defined result to the next. The model supplies content in every phase, and no output of the model can change the order.

The work around the model is done by the runtime (Section 1.2), which here executes the workflow: it calls the model, runs the tests and writes files. The runtime is a component of the program, and it is distinct from run time, the period during which a run happens. The three phases work as follows.

- **Localization.** This phase finds where to edit, narrowing the search in three steps. Each step gives the model a smaller input than the last. First, the model sees only the names of the repository's files and folders and ranks the files most likely to need an edit. The runtime combines this ranking with a similarity search that compares the issue text with pieces of the code, and keeps the three files that rank highest across both. Next, the model sees a **skeleton** of each file, meaning its class and function headers without their bodies, and picks the relevant **elements**, the classes and functions to examine further. Finally, it sees the source code of those elements alone and picks the edit locations. At every step the model ranks and selects from what it is given. It never asks for anything else.
- **Repair.** This phase writes candidate patches. Localization supplies four alternative sets of edit locations rather than a single one. Each set is widened by ten lines on either side, so the model sees the surrounding code, and the model samples ten patches for each set, forty candidate patches in all. Each patch is a **search/replace edit**: the model returns a block of code to find and a block to replace it with. The runtime, and not the model, then writes the change to the file.
- **Patch validation.** This phase chooses one patch. The model samples candidate **reproduction tests**, tests meant to show the issue occurring on the unpatched code. The runtime keeps the candidates that do show it and selects the one generated most often. It then runs the repository's existing tests against every candidate patch and keeps the patches with the fewest failures. Only tests that pass before any change are used, minus any that the model expects the fix to change. Of the patches kept, the runtime keeps those that pass the reproduction test, or all of them if none does. Different samples can produce the same patch once spacing and comments are ignored. The runtime therefore treats each sample as one **vote** for the patch it produced, and selects the remaining patch with the most votes.

In terms of the division between control and content decisions (Table 3), the model makes the content decisions: which files, which elements, which lines, and what each patch says. Every control decision was fixed before the issue arrived, from the order of the phases to the number of samples and the rule that selects the final patch.

In Agentless, the decision to accept a patch is also made outside the model. The final patch is chosen by test outcomes and a vote, and the model's opinion of its own work plays no part. The component that proposes a patch is therefore separate from the component that accepts it. The validator in Section 4.10 applies the same separation.

2.8 Agentless and SWE-agent

SWE-agent is a published agent for the same task. In SWE-agent, the model chooses the next action at every step, the design we study as a reactive loop (Section 3). Agentless and SWE-agent therefore differ in their **control structure**, a system's answer to the question of who decides what runs next: the designer, as

in a fixed workflow, or the model. In the published comparison both systems used the same model, which removes the model as a source of difference between them. For this reason, the comparison comes close to isolating what control structure contributes.

Table 4 reproduces the comparison. Its rows give how each system is controlled, how many of the 300 issues it resolved, and what one issue cost in money and in tokens. The rows to read together are Resolved, Cost per issue and Tokens per issue.

Table 4: Agentless and SWE-agent on SWE-bench Lite, both run with the same model.

SWE-bench Lite	Agentless	SWE-agent
Control	Three phases in a fixed order	The model chooses the next action
Resolved	96 of 300 (32.00%)	55 of 300 (18.33%)
Cost per issue	$0.70	$2.53
Tokens per issue	78,165	498,346

With the same model, Agentless resolved 96 issues and SWE-agent resolved 55 [8, Table 1]. Agentless did so at about a quarter of the cost per issue ($0.70 against $2.53) and with about a sixth of the tokens per issue (78,165 against 498,346). On this benchmark and with this model, the fixed workflow resolved more issues for less money.

The comparison is descriptive, not causal. Using the same model removes one source of variation, but the two systems differ in other ways as well. Their prompts differ, and so do the tools and the interface they give the model. They also choose the final patch differently, since Agentless samples forty candidates and selects one by tests and a vote. Any of these differences may contribute to the result. The comparison therefore describes how these two systems performed. It does not show that control structure alone caused the difference.

2.9 Limits of the comparison

The comparison is limited further by the systems it leaves out and by flaws in the benchmark itself [8, Table 1 and §§6.1–6.2].

- **The leaderboard.** SWE-bench Lite has a public **leaderboard**, a ranking of the systems submitted to it by the number of issues they resolve. Agentless did not lead it. Several closed-source systems scored higher, and the highest, CodeStory Aide, resolved 129 of the 300 issues. These systems published neither their code nor their **trajectories**, the records of the steps each run took. Without them, there is no way to examine how their control structure contributed to their scores, which is the question the comparison is meant to answer.
- **The benchmark.** Some issue descriptions give the answer away or leave out what is needed. In 4.3% of them, about one in twenty-three, the description contains the developer's own patch, so a system can copy the fix instead of finding it. In 10.0%, the description lacks the information needed to solve the issue, so no system can be expected to solve it from the description alone. A filtered set of 249 issues excludes these cases, together with the issues whose descriptions propose a misleading solution.

Taken together with the differences between the two systems (Section 2.8), these limits mean that the comparison does not show that fixed workflows are better in general.

A benchmark **leaks** when the material given to a system contains the answer, as the issue descriptions that include the developer's patch do. Leakage is also a concern in our own setting. OWASP BenchmarkPython, the code base our running example works on, ships a file of expected results that says which of its test cases, the small example programs it is made of, are truly vulnerable. We remove that file before the agent sees the code. If the agent could read the answer key, its results would no longer measure its ability to analyze the code.

Key points

- A fixed workflow is a program in which the designer makes every control decision at design time and the model makes content decisions at run time, so the model may make the hardest choices, such as which file to patch, but never decides what happens next.
- A fixed workflow relies on its designer's prior knowledge for every control decision, so it lacks autonomy in those decisions.
- Fixing the control decisions guarantees shape (the number and order of model calls are known before the run), address (a wrong result can be traced to the step that produced it) and termination (the run ends when its last step returns).
- Architectures that let the model choose further steps replace termination with a budget on model calls, which guarantees that a run stops but not that it reaches an accepted submission.
- Budgeting, checkpointing, transactions and parallel execution are easier to engineer in the fixed stages of a system, because their steps and artifacts are known before the run.
- A fixed workflow cannot look again, notice missing evidence or ask for more, so when the designer's knowledge is incomplete it fails silently, ending as if it had succeeded.
- With the same model, Agentless resolved more SWE-bench Lite issues than SWE-agent at lower cost, but because the two systems differ in more than control structure and the benchmark has known flaws, the comparison does not show that fixed workflows are better in general.

3 ReAct: reactive agents

The fixed workflow of Section 2 reads only what its designer scheduled, so it cannot respond to evidence that the designer did not foresee. The reactive loop removes this limit by letting the model choose the next action after each tool result, while the agent, the alert and the tools stay the same as in Section 2. We define the loop, show how it is built as a graph, and identify who decides each step. We then compare its gains and costs with those of the fixed workflow, review published evidence on where such agents fail, and describe the two stops that the runtime adds.

3.1 What a reactive loop is

A **reactive loop** is an architecture in which the model selects the next action from the percepts so far, and the runtime returns an observation that the next selection reads [9, §2]. The percepts so far are everything the agent has received during the run, such as the alert, file excerpts and search results. As in Section 1.2, the runtime is the code that runs the agent, and an observation is the result of one tool call as the runtime records it, for example the lines that a file read returned.

The runtime keeps the **run's state**, the record of the run so far, which Section 1.8 called the agent's working state: the messages exchanged with the model, the observations, the number of model calls and the run's status. The **status** records whether the run is still running and, if it has ended, how it ended. The loop proceeds in turns. A **turn** is one pass through the loop, from one model call to the next. Every turn consists of the same four steps:

- **Decide.** The runtime makes one model call. A model's **context** is the input it receives in one call, and in the loop it holds everything that has happened so far in the run. The model reads its context and returns a reply that contains its next action: a tool call, a passage of reasoning that ReAct calls a thought, or both. A **tool call** is a request to run one of the agent's tools, such as `read_file`, with arguments.
- **Dispatch.** The runtime checks the tool call against the tool's

signature, the name and the arguments of the tool as we wrote them, and then executes the tool. A malformed call, for example one that names a tool that does not exist, is returned to the model as an observation that describes the error. The run continues instead of crashing. Lecture 4 studies this check in detail.
- **Observe.** The runtime writes the tool's result into the run's state as an observation, and the model never writes one. This division matters for security. The model can write text that looks like a tool result, but such text never becomes an observation.
- **Route.** A predicate that we wrote selects what happens next: another turn, a submission or the end of the run. As in Section 2.2, a predicate is a condition in code, here evaluated over the run's state.

In the graph of the loop (Section 3.2), the routing predicate runs right after the model call, because what it tests is the model's reply. When the reply contains a tool call, the run moves on to the dispatch and observe steps. Once the observation is written, the run returns to the model unconditionally.

ReAct, the published method that this loop follows, rests on one idea: language is part of the action space. An agent's **action space** is the set of actions it can choose from. Without language, the action space of our agent would contain only its tool calls. ReAct adds a second kind of action, the **thought**: a piece of text in which the model reasons about the task. In our running example, a thought might note that the flagged query relies on a helper file, and that this helper should be read before the patch is written.

A thought is an action, but it acts on nothing outside the model's context. It changes the context that the next decision reads, and it leaves the files, and every other part of the environment, unchanged. A tool call, in contrast, either gathers information from the environment, as `read_file` and `search` do, or changes it, as `edit` does. Because a thought stays in the context, the reasoning it records is available to every later decision in the run.

A thought can appear in the same reply as a tool call. A reply that contains a thought and no tool call is routed to submit, because the absence of a tool call is the model's signal that it has finished (Section 3.4).

3.2 The graph of the loop

We build the reactive loop as a graph of three nodes whose edges decide what runs next. As in Section 1.2, each state of `mini-swe-agent`'s state machine becomes a node, and each transition becomes an edge. In code, a node is a function that reads and updates the run's state, and an edge connects a node to another node that may run after it. The word state is used in three ways in these notes. The state of the environment is everything about the world that can matter to the task (Section 1.7). The Controller, Tool and Submit states are modes of the state machine. The run's state is the record of the run that every node reads and writes. Figure 5 draws the graph, and its nodes are the following:

- `controller`. This node is the only place where the model is called. Each visit makes one model call, unless one of the runtime's stops (Section 3.10) ends the run before the call is issued.
- `tools`. This node checks the requested tool call, executes it and writes the observation.
- `submit`. This node runs three mechanical checks on the patch: that it applies, that it touches the flagged line, and that a rescan no longer reports the alert.

The nodes do the work of a run, and the edges decide which node runs next. The architectures in these notes differ in who decides what runs next (Section 2), so the edges are the part of the graph that distinguishes one architecture from another.

The graph has two kinds of edge. An **unconditional edge** is taken every time the run reaches its source node, and the figure draws it solid. A **conditional edge** is taken only when a predicate over the state selects it, and the figure draws it dashed.

There are two unconditional edges. The first, from Start to `controller`, means that every run begins with a model call. The second, from `tools` back to `controller`, is a **back edge**: an edge that leads from a node to an earlier node of the graph, so that the graph contains a cycle and a run can return to a node it has already visited. This back edge returns every observation to the model, which then chooses the next action. The conditional edges are the following:

- **Tool call.** From `controller` to `tools`, when the model's reply contains a tool call.
- **No tool call.** From `controller` to `submit`, when the model's reply contains no tool call.
- **Stopped.** From `controller` to the end of the run, when the runtime has stopped the run because the budget is exhausted or the run is making no progress (Section 3.10).
- **Accepted.** From `submit` to the end of the run, when the checks accept the patch.
- **Rejected.** From `submit` back to `controller`, when the checks reject the patch. The rejection reaches the model as feedback. This edge is the loop's second back edge.

The back edge from `tools` to `controller` is what makes the program a loop. Without it, the model would never see the result of a tool call, so it could not choose its next action based on what it had read. Like the chain, the fixed-workflow version of `mini-swe-agent` (Section 2), the agent could then never choose a read in response to an earlier one.

In code, each edge is declared with one call (Listing 3). The names `START` and `END` stand for the beginning and the end of the run. The first two lines declare the unconditional edges, and the second of them is the back edge from `tools` to `controller`. Deleting that single line would end the loop through the tools, so the model could no longer respond to what it reads.

Listing 3: Illustrative code that declares the edges of the reactive loop and compiles the graph.

```
graph.add_edge(START, "controller")
graph.add_edge("tools", "controller")   # the back edge

graph.add_conditional_edges("controller", after_controller,
   {"tools": "tools", "submit": "submit", END: END})
graph.add_conditional_edges("submit", after_submit,
   {"controller": "controller", END: END})

app = graph.compile()
```

3.3 Conditional edges and compilation

A conditional edge chooses the next node from a fixed list of successors. The **successors** of a node are the nodes that may run after it. We list them when we build the graph, and nothing that happens during a run can add to the list.

In Listing 3, each call to `add_conditional_edges` has three parts: a source node, a **routing function**, and a dictionary. The routing function reads the state and returns a key. The dictionary maps each key to the node that runs next.

The successors of `controller` are `tools`, `submit` and the end of the run. The successors of `submit` are `controller` and the end of the run. Each entry in a dictionary appears as one dashed edge in Figure 5, so three dashed edges leave `controller` and two leave `submit`.

Because the routing function returns a key rather than a node, it cannot send the run to a node we did not list. Whatever key it returns, the run can only move to a node named in the dictionary, and a key that is not in the dictionary causes an error. When we say

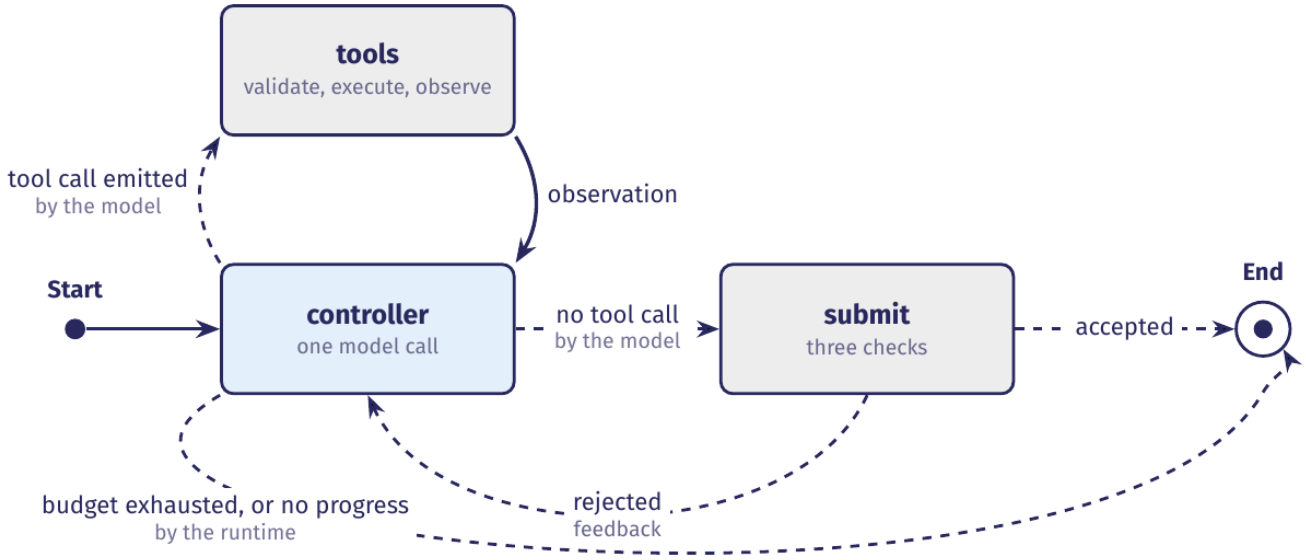


Figure 5: The graph of the reactive loop. Only the controller node calls the model. Solid edges are always taken, and dashed edges are taken when a predicate over the run's state selects them. The grey labels *by the model* and *by the runtime* name who supplies the value that selects an edge. The solid edge from tools back to controller makes the program a loop.

that the model chooses what happens next, we mean something narrow: it chooses among successors we named in advance.

Once the nodes and edges are declared, the graph is **compiled**, that is, turned into a runnable object (the last line of Listing 3). Compilation happens once, before the program reads any alert, and the same compiled object runs every alert.

A **path** is the sequence of nodes that one run actually visits. Two runs that take different paths still run the same compiled graph. Their paths differ because the predicates evaluate to different values on different states: for example, the model requests a tool in one run and submits in the other. The structure of the graph does not change during a run or between runs.

If we want runs to behave differently in some situation, we do not rebuild the graph between runs. We add a predicate that tests for that situation and write it into the graph before it is compiled. The compiled graph then stays the same from one run to the next.

3.4 Who picks the edge

The model selects the node that follows `controller` in exactly one way: by including a tool call in its reply, or by leaving it out. The routing function after `controller` shows this mechanism. In code, it looks like this (Listing 4):

Listing 4: Illustrative code for the routing function after the controller node.

```
def after_controller(state) -> str:
    if state["status"] != RUNNING:
        return END
    if state["messages"][-1].tool_calls:
        return "tools"
    return "submit"
```

The function makes two tests, and each tests a value that a different actor supplied:

- **The runtime's test.** Only the runtime writes the status field. If the status is no longer `RUNNING`, the runtime has stopped the run (Section 3.10), and the function returns `END`.
- **The model's test.** If the model's last message contains a tool call, the run goes to `tools`. Otherwise it goes to `submit`. A reply without a tool call is the model's signal that it has finished.

The model has no other way to choose what happens next. Its text affects routing only through the presence or absence of a tool call. Every other branch is selected by a value that the runtime supplies. That value is either the status, when the runtime has stopped the run, or the result of the submit checks, which decides between acceptance and rejection.

The runtime records each routing decision in the run's **trace**, the log of events it writes during a run. The event it writes for a routing decision is a **routing event**. Each routing event has a `by` field that names who supplied the value the predicate tested. It is `by="model"` when the model's output, a tool call or its absence, selected the branch. It is `by="runtime"` when a field the runtime owns selected it, for example the status set by a stop or the result of the submit checks. In both cases, the predicate itself was written by us. Reading the `by` field across a trace shows which decisions in a run came from the model and which came from the runtime.

3.5 The model's autonomy increases

The reactive loop moves several control decisions from the designer to the model. Table 5 lists the decisions of a fixed workflow (Section 2) and shows who makes each one in the two architectures. In the fixed workflow column, every control decision is ours and is made at design time. In the reactive loop column, the model makes the decisions about which operation runs next, whether another runs after it, and which artifacts a step touches. The last row is a content decision, which the model makes in both architectures.

Each row has the following meaning in the loop:

- **Which operation runs next.** The model chooses it at run time by deciding whether to call a tool and which one.
- **Whether another runs after it.** The model keeps the run going by requesting further tools, until it stops requesting them or the budget is exhausted.
- **How many model calls.** Nobody fixes the number in advance. The budget we set only bounds it from above.
- **What ends the run.** A predicate we wrote ends the run, as in the fixed workflow.
- **Which artifacts a step touches.** The model chooses them through the arguments of its tool calls, for example which file to read or which lines to edit.
- **What a model call returns.** The model still decides it.

Table 5: Who makes each decision in a fixed workflow and in a reactive loop. In the reactive loop, the model takes over the choice of the next operation, whether another follows, and which artifacts a step touches.

Decision	Fixed workflow	Reactive loop
Which operation runs next	Us, at design time	The model, at run time
Whether another runs after it	Us, at design time	The model, until the budget
How many model calls	Us, at design time	Bounded by the budget
What ends the run	Us, at design time	A predicate we wrote
Which artifacts a step touches	Us, at design time	The model, by using tools
What a model call returns	The model	The model

Even where the model gained a decision, it chooses among options we defined. We still write the tool signatures, the **state schema** (the fields the state contains and their types), the routing predicates and the budget. The model selects its next action from a set of options we fixed in advance. A reactive loop is still a program that we write. What we no longer write down is the order of its operations.

An agent lacks autonomy to the extent that it relies on its designer's prior knowledge rather than on its own percepts (Section 2.3). By this measure, the reactive loop is more autonomous than the fixed workflow in the decisions the model has taken over. The choice of the next action, for example, now relies on the agent's percepts, the results it has observed during the run, rather than on the designer's prior knowledge alone. The decisions that remain ours, such as the budget and what ends the run, still rely on prior knowledge.

3.6 What the loop gains

Letting the model choose the next action gives the agent three benefits, which we call reach, proportion and recovery:

- **Reach.** The agent can take an action whose purpose is to gather information, and then use what it learned in its next decision. Suppose the flagged line calls a helper function defined in another file. The model can read that helper and only then decide how to patch. The chain has no step in which to make this choice, because its reads were scheduled before the run.
- **Proportion.** The cost of a run follows the difficulty of the alert. An easy fix takes only a few turns, and a hard one takes more. The chain makes the same number of model calls for every alert, so an easy alert costs as much as a hard one.
- **Recovery.** An unhelpful observation is not final, because there is a next turn in which to try something else. For example, a search that returns nothing can be followed by a different search.

All three benefits come from a single change to the graph: the back edge from `tools` to `controller`. Because of that edge, the model sees each tool result and chooses the next action. No new node is needed to give the agent a second look, a cost that grows with the difficulty of the task, or a way to recover from an unhelpful result.

The chain has no such edge. If its single read returns the wrong part of the code, the error passes unchanged through every later step and into the patch.

3.7 What the loop costs

The reactive loop gives up the three guarantees of the fixed workflow, shape, address and termination (Section 2.4). They are lost together, because each of them depended on the designer fixing the order of the steps before the run:

- **Shape.** In a fixed workflow, the number and order of model calls are known before the run. In the loop, the number of model calls depends on the model's choices, so we cannot know the cost of a run before it starts. The budget gives only a worst case. It is an upper limit and not an estimate, so for each alert we must reserve the cost of the whole budget, however few calls the alert turns out to need.
- **Address.** In a fixed workflow, a wrong result can be traced to the step that produced it. In the loop, a failure is spread over many turns, each of which looks reasonable on its own. Finding the turn where the run went wrong means reading the whole trace.
- **Termination.** In a fixed workflow, the run ends when its last step returns. The loop has no last step. A run ends when a predicate we wrote ends it: either the submit checks accept a patch, or a limit we imposed, such as the budget, stops the run. A run stopped by a limit ends without an accepted patch. It has stopped, but it has not finished.

The three losses share one missing capability: the loop has no measure of progress. **Progress** here means movement toward an accepted patch. The loop cannot tell a turn that advanced the investigation from a turn that wasted a model call. Apart from the stall check (Section 3.10), which notices only results that repeat exactly, the only thing the runtime measures about a run is the number of model calls it has made, and the budget is a limit on that number. A run that reaches the budget has paid for every model call it was allowed and still ends without an accepted patch.

A plan would give the run a target against which each turn can be measured. The planner and the validator are designed to supply this missing capability (Sections 4 and 4.10).

3.8 Where the failures occur

In a published study, a fixed workflow and two reactive loops failed at different stages of a run [3]. The study, available as a preprint, examined 150 issues on which at least one of three issue-solving tools failed, programs that take an issue report and a repository and try to produce a patch. The failures come from runs on SWE-bench Verified, a benchmark of software issues of the same kind as SWE-bench Lite (Section 2.7). Each failure was annotated by hand with the stage of the run in which it appears.

All three tools used the same **backbone model**, the language model that a tool calls to make its decisions. One tool, Agentless-1.5, is a fixed workflow. The other two, OpenHands-CodeAct-2.1 and Tools + Claude 3.5 Sonnet, are reactive loops. Table 6 gives, for each tool, the stage the study associates with its failures and the share of that tool's own failures found at that stage. Each share is a fraction of one tool's failures, not of all issues.

Agentless-1.5, a version of the Agentless workflow of Section 2.7, fails most often at its first decision. Just over half of its failures (51.3%) involve an error in localization, the first phase, which decides where in the code to edit. No later phase revisits that decision, so the rest of the run builds on a mistake it cannot detect.

About half of each reactive loop's failures appear in the study's iteration and validation stage, which covers the repeated turns that the model drives. The fixed workflow has no such stage, because its phases run once, in an order its designers fixed. Agentless does validate its patches, but in one fixed phase that the model does not direct. The mechanism that gives the loop reach, proportion and recovery is also where its new failures occur.

The comparison is descriptive. Holding the model fixed removes one source of variation, but the tools also differ in their prompts and their interfaces. The study therefore does not show that the architecture alone causes the difference. It also does not show which tool fails more often, because each share is a fraction of a single

Table 6: Where the failures of three issue-solving tools appear, with the share of each tool's own failures at that stage. All three tools used the same backbone model.

Tool	Architecture	Where the failure appears	Share
Agentless-1.5	Fixed workflow	Localization	51.3%
OpenHands-CodeAct-2.1	Reactive loop	Iteration and validation	49.1%
Tools + Claude 3.5 Sonnet	Reactive loop	Iteration and validation	51.7%

tool's failures. What it shows is where each tool's failures occur.

3.9 Shapes of failure inside the loop

In the same study, roughly 65% of the failures trace back to flawed reasoning by the model, for example persisting with an approach that keeps failing [3]. The study also names the failure modes it found at each stage. We call the following failure modes of the iteration and validation stage, where about half of each reactive loop's failures appear, **failure shapes**. The term is unrelated to the shape guarantee of Section 2.4. Each failure shape motivates a mechanism that we study later:

- **Non-progressive iteration.** The agent generates nearly identical patches again and again, with trivial differences, while the defect remains. The loop keeps running without getting closer to a fix. A plan addresses this shape by giving the run a target against which each turn can be measured (Section 4).
- **Validation retreat.** The agent edits the failing test instead of the code, so that the test passes while the defect remains. The component that proposed the patch has, in effect, graded its own work. A validator addresses this shape by giving the decision to accept a patch to a component that did not write it (Section 4.10).
- **Context amnesia.** The agent loses track of the original objective over a long interaction. Lecture 3 proposes a reflection step for this shape. After the validator rejects a patch, the step writes a description of what went wrong, and a new attempt starts. The new attempt clears the plan and the patch but keeps that description.

In general, the loop cannot detect any of these shapes while it runs, because it has no measure of progress (Section 3.7). The stall check (Section 3.10) catches only one narrow form of non-progressive iteration, the form in which the results repeat exactly.

The study also measured how long the runs of the two reactive loops last. Failed runs take on average about 3.5 times as many steps as successful ones, and the study links this to failures that trap the agent in a loop. The study counts run length in **rounds**, where a round is one turn of the loop. For OpenHands-CodeAct-2.1, 80% of the successful runs end within 25 rounds, but 54 rounds pass before 80% of the failed runs have ended. Failed runs therefore last longer and consume more of the budget.

These counts can be computed only after the runs have ended, when we know which runs failed. During a run, the loop has no signal that separates a run heading toward an accepted patch from a run that repeats itself without progress.

3.10 The two runtime stops

Because the loop cannot recognize a failing run, the runtime has two stops. A **stop** is a check in the runtime that ends a run when its condition holds, whatever the model would do next. We write both stops, and the `controller` node checks both before it issues the model call. In code, the two checks look like this (Listing 5):

Listing 5: Illustrative code for the two stops, at the start of the controller node.

```
if state["model_calls"] >= budget:        # budget = 40
    return {"status": BUDGET_EXHAUSTED}
if stalled(state["observations"], limit=3):
    return {"status": NO_PROGRESS}
# ... only now is the model called
```

The two stops work as follows:

- **The budget.** When the number of model calls in the run has reached the budget, 40 in our agent, the run ends with the status `budget_exhausted` (the constant `BUDGET_EXHAUSTED` in the code).
- **The stall check.** When the last three observations are identical, the run ends with the status `no_progress` (`NO_PROGRESS` in the code). The function `stalled` makes this comparison, and `limit=3` sets how many observations it compares.

Each stop sets the status field, and the routing function after `controller` (Listing 4) then sends the run to its end.

The checks run before the model call so that they act as controls. A check is a **control** when it prevents an unwanted outcome, here a model call beyond the limit. A check placed after the call would detect that a limit had been passed only once the extra call had been made and paid for. It would record the overrun instead of preventing it, so it would be a report after the fact rather than a control.

The stall check is placed before the call for the same reason: a run that has stopped making progress ends before it pays for another model call. Because the budget check runs before the call, a run never makes more model calls than the budget allows. A run stopped by the budget records no further model call after its last routing event.

The stall check compares results rather than calls. A simpler rule would stop the run when the model repeats the same call, but the two rules are not equivalent. A model that changes its `search` pattern slightly on each attempt never repeats a call, yet it can make no progress at all. For example, a model that tries six different search patterns, each of which returns no matches, never repeats a call. A same-call rule would not stop it, and only the budget would end its run. Every one of those searches returns the same empty result, which is exactly the condition the stall check tests.

Comparing results also covers the case of identical calls. The task environment is deterministic, so identical calls on files that have not changed return identical results (Section 1.7).

The stall check has a clear limit. It catches one form of non-progressive iteration, the form in which the results repeat exactly. A run whose results keep changing, even trivially, is not stopped by the check. The loop cannot detect the general failure shapes of Section 3.9 while it runs, so such a run, if it never produces an accepted patch, ends only when it reaches the budget.

3.11 An example run

An example run of the loop on the worked alert shows the four steps of a turn, and the benefits of reach and proportion, in practice. The alert flags a SQL query built by string interpolation (Section 1.3). In each of the first three turns, the model requests a tool, the runtime executes it and writes the result as an observation, and the run returns to the model. The run takes four turns:

1. The model reads the region of the flagged file around the alert.
2. The model reads a helper file that the flagged query uses, which the alert does not name. This is reach: the chain, which reads

only the region around the flagged line, could not have chosen this read. The run in Figure 2 did not make this read, because two runs on the same alert can take different paths (Section 1.7).
3. The model edits the flagged file so that the query text contains a placeholder and the value is passed separately as a bound parameter.
4. The model replies without a tool call, which signals that it has finished. The `submit` node runs its checks: the patch applies, it touches the flagged line, and the rescan is clean. The runtime ends the run with the status `accepted`.

The run was accepted after four model calls against a budget of 40, so neither stop was close to ending it. This is proportion: an easy fix took only a few turns. The model made every routing decision in the run except the last: it requested a tool in the first three turns and replied without one in the fourth. The runtime made the last decision, the acceptance.

Key points

- A reactive loop lets the model choose the next action after each observation, and every turn has four steps: decide, dispatch, observe and route.
- ReAct makes language part of the action space: a thought is an action that changes only the model's context, while a tool call reads or changes the environment.
- The loop is one graph, compiled once, in which each conditional edge chooses among successors fixed when the graph is built, and different runs take different paths because the predicates evaluate differently on different states.
- The model chooses the next node only by including a tool call in its reply or leaving it out, and the runtime supplies the value that selects every other branch.
- The back edge from tools to controller creates the loop and gives it reach, proportion and recovery, and the loop loses shape, address and termination because the order of its steps is no longer fixed before the run.
- In a study with one backbone model, about half of the fixed workflow's failures appeared at localization and about half of each reactive loop's in iteration and validation, but the study does not show that architecture is the cause.
- The runtime checks its two stops, the budget and the stall check, before each model call so that they prevent an unwanted call instead of recording it, and the stall check catches only the non-progressive iteration whose results repeat exactly.

4 Planning and validation

The reactive loop of Section 3 has two weaknesses. It has no measure of progress, and the checks that accept its patches do not show that the weakness is gone, which the performance measure requires. This section adds one mechanism for each weakness to `mini-swe-agent` and builds both into a single graph. It also examines the evidence on how an agent should choose its next action and on how reliably a model can judge its own work.

A **planner** is a node that writes the steps the run intends to take into the run's state, and revises them when the run stalls. A run **stalls** when its last three tool calls return the same observation, the condition that the stall check of Section 3.10 detects. A **validator** is a node that decides whether to accept a patch that has passed the mechanical checks. It is separate from the component that proposed the patch. The agent, the alert and the tools are those of the earlier sections, so the only change is the architecture.

4.1 What the reactive loop cannot answer about itself

The reactive loop records everything it has seen, but nothing in it records what the run set out to do. As a result, the program cannot distinguish a turn that moved the run closer to an accepted patch from a turn that only produced another observation. A turn is one pass through the loop (Section 3.1).

The loop can detect that an observation is new. A new observation, however, does not show that the run has moved closer to an accepted patch, so the loop has no measure of progress. The gap appears as questions that the loop cannot answer about its own run:

- **Is this part of the task finished?** Nothing states what finishing it would require, so the loop cannot tell.
- **Where is the run heading?** Halfway through a run, the trace holds the model's thoughts so far, but none of them states where the run is heading. Neither the program nor a reader can tell what the remaining turns are meant to achieve.
- **What should change when part of the task proves impossible?** There is no remaining plan to revise, because no plan was ever written.

Each question can be answered only if something was written down before the turns it asks about. Measuring progress needs a goal to compare each turn with. Showing where the run is heading needs a statement of intent. Changing course needs a list of remaining steps that can be replaced.

A **plan** is a list of what the run intends to achieve, written into the run's state before the work begins. Each entry is a **plan step**. The graph in Figure 7 calls plan steps subgoals. Because the plan is in the state, the program can read it and later replace it. A plan is the smallest addition that answers all of these questions. The model still chooses every action, and the run gains one record that states what those actions are for.

4.2 Open-loop and closed-loop planning

Planners differ in how they use the observations that the environment returns after each action [6]. A planner is **closed-loop** when these observations return to it, and **open-loop** when they do not. A closed-loop planner may use an observation only to choose its next action, or it may use it to rewrite the rest of its plan. The open-loop planner and these two kinds of closed-loop planner give three shapes of planning (Figure 6).

In the figure, a_t is the action taken at step t, o_{t+1} is the observation that comes back after it, and T is the last step of the plan. The three shapes are as follows:

- **Open-loop.** The planner sends its whole sequence of actions, a_1 to a_T, at once, and nothing comes back. Nothing changes in response to the environment.
- **Implicit closed-loop.** The planner sends one action at a time, and every observation comes back to it. The plan stays as it was first written, and only the next action changes.
- **Explicit closed-loop.** Every observation comes back, and the planner may use it to revise the plan itself. AdaPlanner, the example in the figure, sorts each observation by whether it agrees with what the plan predicted. When an observation violates the prediction, the planner strikes out the remaining actions, a_{t+1} to a_T, and writes new ones, a'_{t+1} to a'_T, in their place. The figure draws the struck-out actions in red and the new ones in green.

Both closed-loop shapes receive every observation. They differ in what an observation may change, and only the explicit closed-loop shape uses it to change the plan itself.

AdaPlanner, a planning method for language-model agents, is drawn inside the dashed box as an example of the explicit shape. It sends an observation that agrees with the prediction to an in-plan refiner, which extracts information from the observation and leaves the plan unchanged. It sends an observation that violates the prediction to an out-of-plan refiner, which leads the planner to replace the rest of the plan. The skill memory in the figure is part of AdaPlanner and plays no part in the distinction between the three shapes.

Table 7 summarizes the shapes. The column to compare is the last

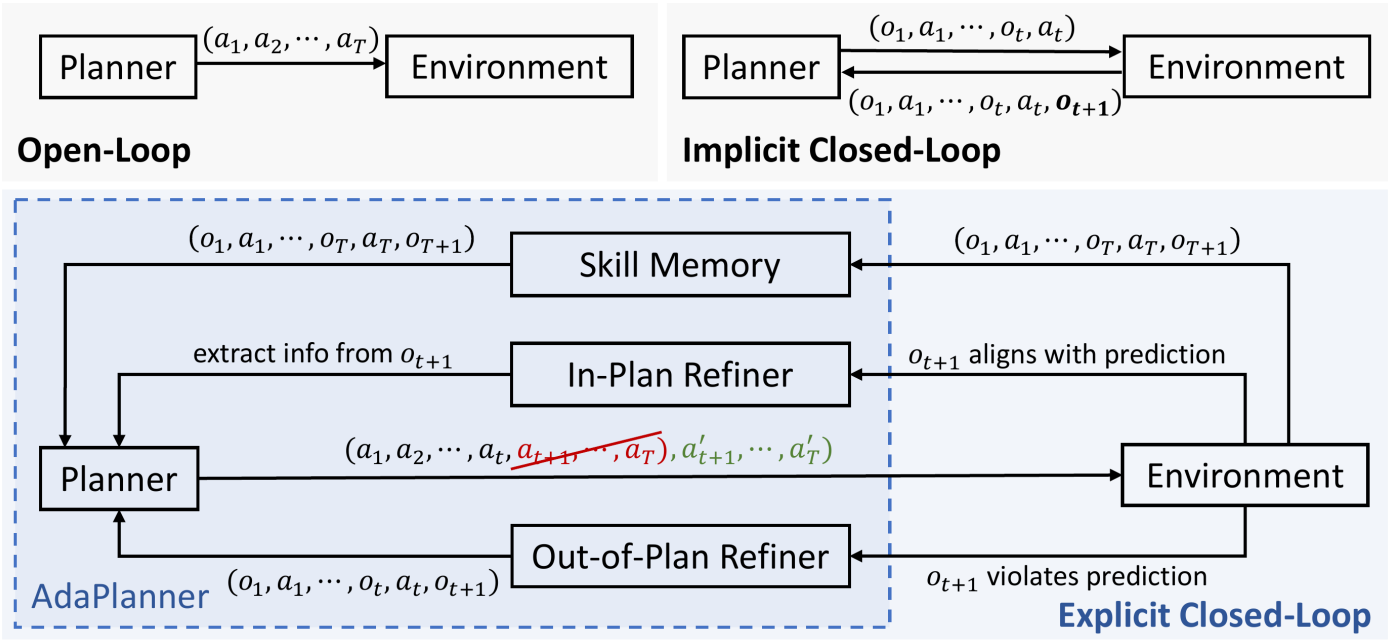


Figure 6: Three shapes of planning: open-loop (top left), implicit closed-loop (top right) and explicit closed-loop (bottom), with AdaPlanner as an example of the third. Figure 1 of Sun et al., AdaPlanner, NeurIPS 2023, reproduced for teaching.

one, which says what a planner may change when an observation arrives.

Table 7: What each shape of planning receives from the environment, and what it changes in response.

Planner	What comes back	What changes
Open-loop	Nothing	Nothing
Implicit closed-loop	Every observation	The next action only
Explicit closed-loop	Every observation, used to revise the plan	The rest of the plan

Judged by what an observation may change, the reactive loop of Section 3 resembles an implicit closed-loop planner without a plan. Like that planner, it receives every observation and uses it only to choose the next action. Unlike that planner, it has no plan to keep or to revise.

The loop also only ever adds to the model's context, the messages the model reads at each call. Every observation and every thought is appended, and nothing is removed. A plan revision works differently. It replaces the rest of the plan held in the state, so the plan the run now follows no longer contains the steps that were replaced.

4.3 The graph of the planner and the validator

We add both mechanisms to `mini-swe-agent` as one graph with five nodes (Figure 7). Two of the nodes are new: `plan`, the planner, and `validate`, the validator. The model is called in three nodes, `plan`, `controller` and `validate`, which the figure draws in blue.

A run passes through the nodes as follows:

- **Plan.** Every run starts at `plan`. The planner writes a plan at the start of the run and revises it after a stall, then passes the run to the controller.
- **Controller.** The `controller` node makes one model call. If the reply contains a tool call, the run goes to `tools`. If it contains none, the run goes to `submit`.
- **Tools.** The `tools` node checks the requested tool call before running it (the dispatch step of Section 3.1), executes it and records the observation. The word "validate" inside this node in the figure refers to that check of the tool call. It is unrelated to the validator. The run then returns to the controller, or to the planner if the run has stalled.
- **Submit.** The `submit` node runs the three mechanical checks of the reactive loop: the patch applies, it touches the flagged line, and the rescan is clean (Section 1.3). The node cannot accept the patch. If a check fails, the run returns to the controller with the reasons. If all three pass, the run goes to `validate`.
- **Validate.** The `validate` node is the only node that can accept a patch. Its verdict either accepts or rejects the patch, and the run ends after the verdict in both cases.

Two edges are unconditional, which means the graph takes them every time it reaches their source. One starts every run at the planner, and the other ends the run after the validator. They are the solid edges in the figure. Every other edge is conditional. For each conditional edge, a routing function that we wrote reads the state and returns the name of the next node. Each dashed edge is one destination that such a function can return.

Two consequences follow from these edges. In the reactive loop, a run started at the controller. With a planner, every run begins by planning, and the only way a run can end accepted is through the validator.

A run ends without acceptance in three cases: the validator rejects the patch, the budget is exhausted, or the run stalls a second time (Section 4.5). The figure draws the first case as the edge from `validate` to the end.

The figure does not draw the two runtime stops, the budget and the second stall. In those cases the runtime sets the run's status, and a routing function then sends the run to its end. For the same reason, the edge from `plan` to `controller` is dashed. Its routing function sends the run to the controller unless a stop has already fired.

Several edges are new, but the curved edge from `tools` back to `plan`, labeled "nothing new came back", is the one that changes the loop itself. It is the path by which a stalled run reaches the

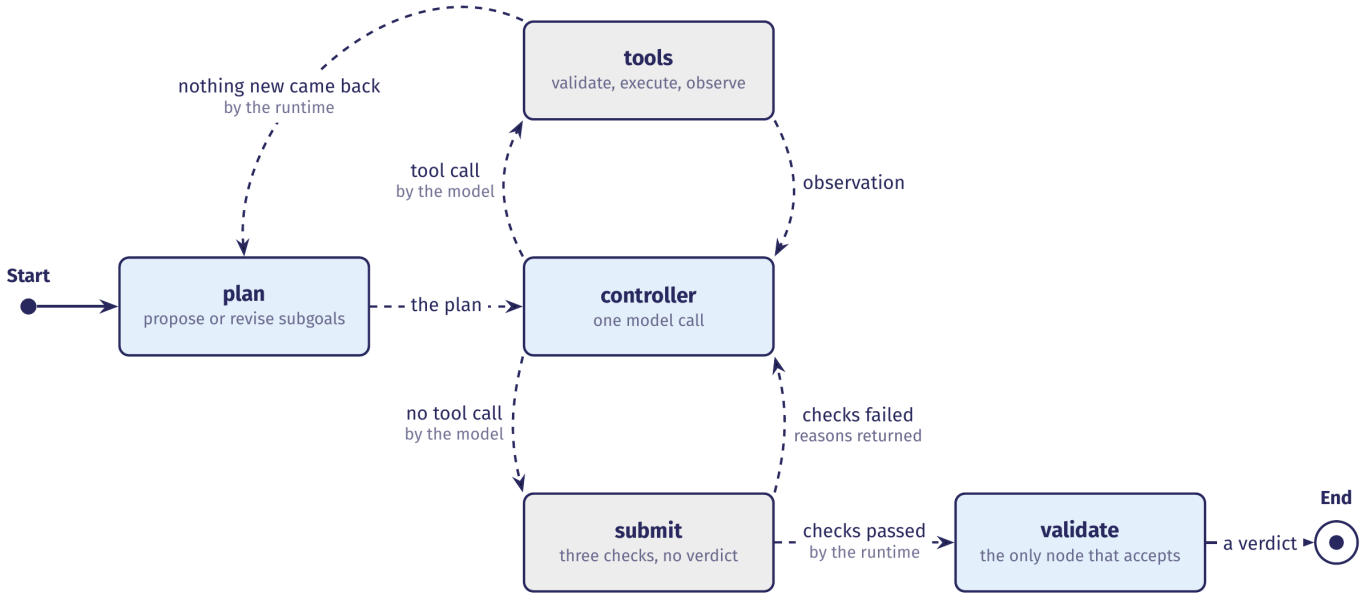


Figure 7: The graph of the planner and the validator. Blue nodes call the model. The two solid edges are unconditional, and each dashed edge is one destination of a routing function we wrote.

planner (Section 4.4).

4.4 The edge from `tools` now has two destinations

With a planner, the edge that leaves `tools` becomes conditional, and its second destination is the planner. In the reactive loop, this edge was unconditional. Every observation went straight back to the controller, which is what made the agent reactive. In code, the routing function after the tools node looks like this:

Listing 6: Routing after the tools node (illustrative).

```
def after_tools(state) -> str:
   if state.get("failed_step"):
      return "plan"
   return "controller"
```

The function reads one field of the state, `failed_step`. If the field is set, the run goes to `plan`, which revises the plan. Otherwise the run goes to `controller`, exactly as in the reactive loop. The runtime sets the field when the run stalls for the first time (Section 4.5).

This edge changes what happens to a stalled run. In the reactive loop, a stalled run ended at once with the status `no_progress`. That status still exists, but a run now reaches it only after one revision of the plan has been tried.

4.5 How the runtime decides that the plan has failed

The runtime, and not the model, decides that the current plan has failed. It uses the stall check of the reactive loop (Section 3.10): a run has stalled when three consecutive tool calls return the same observation. The check catches one form of non-progressive iteration, the form in which the results repeat exactly. Non-progressive iteration in other forms, and the other failure shapes of Section 3.9, go undetected while the run is going on.

What is new is the way a stall leads to a revision:

- **The runtime notices.** In the reactive loop, the controller ran the stall check before each model call and ended the run when it fired. With a planner, the `tools` node runs the check as soon as it has recorded the new observation. When the last three results are identical, it writes one field, `failed_step`, instead of ending the run.
- **The routing function reads one field.** A routing function receives the state and returns the name of the next node. It cannot write to the state, so a judgement made inside it would leave no record. The judgement is therefore made in the `tools` node and recorded in `failed_step`, and the routing function only tests whether that field is set.
- **The planner gets one chance.** After a revision, the stall check starts counting again. If the run stalls a second time, it ends with `no_progress`, the same status the reactive loop used.

Counting again needs care. The runtime only appends to the list of observations and never removes an entry. Immediately after a revision, its last three entries are therefore still the identical results that caused the revision. A stall check that read the whole list would count those old results against the new plan, so the run could end before the new plan had been tried.

The runtime avoids this with a **high-water mark**, a field that records a position in a list. The field `replanned_at` records how many observations existed when the plan was revised. The stall check then reads only `observations[replanned_at:]`, the observations recorded since the revision. The stall that caused a revision therefore cannot also count against the revised plan.

The same field also enforces the limit of one revision. Once `replanned_at` has been set, the one permitted revision has been used, so the next stall ends the run instead of returning to the planner.

Only the runtime writes `failed_step`. The decision that a plan has failed therefore rests on what the tools returned, rather than on the model's opinion of its own plan.

A single failed tool call is a different event from a stall, and it does not set `failed_step`. The runtime reissues a failed call once, without calling the model, because a transient failure is not evidence about the plan.

In the trace, the routing event for the edge to `plan` records `by="runtime"`, because the value it tested is a field the runtime owns. As with every edge, the routing function itself was written by us.

4.6 What a planner changes compared with a reactive loop

Adding a planner changes the answers to four questions about how a run is controlled. Table 8 sets the reactive loop beside the loop with a planner, one question per row. In each row, the last column shows what the plan adds. It gives the model's choices a scope, records the run's intent, states what would show each step is done, and allows a second chance after a stall.

Table 8: Questions about how a run is controlled, answered for a reactive loop and for the same loop with a planner.

Decision	Reactive loop	With a planner
Who chooses the next operation?	The model, for the whole task	The model, guided by the current plan step
Where is the intended course recorded?	Nowhere	In the plan, a field of the state
What would show that part of the task is done?	Nothing is stated	The evidence named in the plan step
What happens when the run stalls?	The run ends	The plan is replaced, once

The model still chooses the next operation, now guided by the plan step it is working on rather than by the task as a whole. The run's intended course, which nothing recorded before, is now recorded in the plan. Each plan step names the evidence that would show it is done. The reactive loop stated no such condition. When the run stalls, it no longer ends at once, because the plan is replaced one time first.

Taken together, these changes mean that the run's aim is written down before the work. The controller's model, and anyone reading the trace, can then measure each turn against a stated goal, rather than only against whether it produced something new. The runtime still detects a failing plan only through the stall check, which does not read the plan (Section 4.5).

4.7 What one plan step records

Each plan step is a small record with three fields: an `id`, a `goal` and the `evidence`. Listing 7 shows one plan step for the worked alert, as the planner records it in the state. The flagged line of that alert is line 46.

Listing 7: One plan step for the worked alert.

```
{"id": 2, "goal": "read the helper the flagged line calls",
  "evidence": "the value that reaches line 46"}
```

In this step, the goal is to read the helper function that the flagged line calls. The evidence is the value that reaches line 46, so the step is done when the run has seen which value reaches that line. Both the fields of the record and the way the plan reaches the controller are deliberate:

- **The goal and the evidence are separate.** The goal says what to do. The evidence says what would show that it was done, which is the part the reactive loop had nowhere to record.
- **The plan arrives as input.** The plan reaches the controller as a human message, that is, a message in the role the model reads as input from a user. The plan was written by a separate model call, the planner, and the human message records that origin. In the controller's conversation, the plan therefore appears as an instruction the controller was given. It does not appear among the controller's own replies, where it would look like work the controller had planned itself.

The only tool offered to the planner's model call is `propose_plan`, through which the model returns the plan as a list of goals, each paired with the evidence that would show it is done. The planner numbers the steps to give each its `id`.

If the planner's model call does not call `propose_plan`, the planner returns no plan steps. The run then continues without a plan, as the reactive loop would. A planner that fails to produce a plan therefore cannot end the run. The routing after the planner sends the run to the controller unless a stop such as the budget has already fired.

4.8 Choosing the next action: evidence on early errors

A planner looks beyond the next step. A study of action selection measured what looking ahead is worth when an agent chooses its next action. It found that the way an agent scores its candidate actions affects how early its runs go wrong and how often they recover [7, Table 3]. The study is a preprint.

The comparison changed only the method that chooses the next action and kept the rest of the agent fixed. It was run in two agent frameworks, each held fixed, and repeated with four language models of different sizes. The tasks were three question-answering benchmarks. In each, an agent answers a question by taking a sequence of steps through a knowledge graph, a database of linked facts.

All four methods pick the next action from a set of candidates. They differ in what they score each candidate against:

- **One step at a time.** Each candidate action is scored on its own, by how good it looks now, and the best one is taken.
- **Beam search.** Several of the best-scoring candidates are kept at each step instead of one, but they are scored in the same way.
- **Lookahead.** Each candidate is scored by the futures it leads to. The method simulates the steps that would follow and evaluates where they end.
- **Lookahead with value propagation.** The value of each simulated future is passed back to the earlier choices that led to it. Evidence from late in a simulation can therefore revise a choice made early.

A **dead end** here is an action that the researchers identified in each task in advance as a trap: it ranks among the best by how good it looks now, but it leads the run away from good solutions. The study reports three measures of how runs fail:

- **Dead end on move one.** This is the share of runs whose first action is such a trap. Lower is better.
- **Step of first error.** This is the average step at which a run first takes a wrong action. Higher is better, because the error comes later.
- **Recovers after it.** This is the share of runs that recover after their first error. Higher is better.

Table 9 gives the results, aggregated over the three benchmarks. The rows to compare are the first, choosing one step at a time, and the last.

Lookahead with value propagation does best on all three measures. Compared with choosing one step at a time, it reduces dead ends on the first move from 55.6% to 17.8%, to about a third. It delays the first error from step 1.6 to step 3.2, twice as far into the run. It raises recovery after the first error from 5.4% to 29.7%, more than five times as often.

A **frontier model** is one of the largest and most capable models available at a given time. The same study reports, in results not reproduced here, that a model with eight billion parameters using

Table 9: Where runs go wrong under four methods of choosing the next action, aggregated over three benchmarks. Lower is better in the first column of numbers, and higher is better in the other two.

How the next action is chosen	Dead end on move one	Step of first error	Recovers after it
One step at a time	55.6%	1.6	5.4%
Beam search	71.9%	2.0	11.4%
Lookahead	23.6%	2.8	22.4%
Lookahead with value propagation	17.8%	3.2	29.7%

lookahead with value propagation often outperforms GPT-4o, a frontier model, when GPT-4o chooses one step at a time. In that comparison, a better method of choosing actions made up for a much smaller model.

The evidence has limits. The tasks are question answering over a knowledge graph, not program repair, so the numbers are not rates we should expect from our agent. Our planner also does not simulate futures. What carries over is the pattern: scoring an action by what follows it, rather than by how it looks now, made early errors rarer and recovery more likely.

4.9 Why wider search does not fix myopia

A score is **myopic** when it rates an action by how good it looks now, without regard to what the action leads to. Searching more widely does not fix a myopic score. In Table 9, beam search considers more candidates than choosing one step at a time. It does better on the step of the first error (2.0 against 1.6) and on recovery (11.4% against 5.4%). Yet its first action is a dead end more often, in 71.9% of runs against 55.6%.

The explanation lies in the score that each method uses:

- **Beam search keeps the myopic score.** It ranks more candidates, but it ranks them with the same next-step score that misled the method that chooses one step at a time.
- **Lookahead changes the score.** What improved every measure in Table 9 was evaluating an action by what follows it rather than by how good it looks now.

When an agent's runs go wrong early and do not recover, we should therefore first examine how it scores its candidate actions, and only then how many candidates it considers.

4.10 Why `submit` can no longer accept a patch

The second mechanism added to `mini-swe-agent` is the validator, the node `validate`, which decides whether to accept a patch. It calls the model, and it is the only node that can end a run with an accepted patch. As a result, the `submit` node can no longer accept a patch. It can only send the patch back to the controller or pass it on to the validator.

In the reactive loop, acceptance was already separate from the controller, the component that proposes the patch. The `submit` node accepted a patch when its three mechanical checks passed, and no model takes part in those checks. What changes with the validator is the routing after `submit`. In code, it looks like this:

Listing 8: Routing after the submit node (illustrative; the branch for the runtime's stops is omitted).

```
return "validate" if checks.passed else "controller"
```

A patch that passed all three checks goes to `validate`. Every other patch goes back to `controller`. No branch accepts the patch and ends the run, so `submit` can no longer end a run accepted.

The checks and the validator therefore work in sequence, and they are not alternatives:

- **The checks are a gate.** A patch that fails them never reaches the validator. It goes back to the controller with the reasons for the failure.
- **The validator is the only accepting exit.** It is the one node that can end a run accepted, and every patch it sees has already passed the mechanical checks.

The validator answers a question that the checks cannot. The checks establish that the rescan is clean. The validator is asked whether the patch is a correct fix, given that clean rescan. A clean rescan shows that the scanner no longer recognizes the flagged pattern. It does not show that the untrusted value can no longer reach the SQL query (Section 1.5). Asking that second question is the purpose of the node.

4.11 Why the validator never reads the conversation

The validator is given the evidence of the run and is kept away from the conversation. The state holds two records of what happened. The `messages` field holds the conversation with the model, including the model's own account of what it found. We call the controller's model, which proposed the patch, the **proposer**. The `observations` field holds what the tools actually returned, one record per tool call, written by the runtime.

The validator's inputs are chosen accordingly:

- **The alert, the patch and the check results.** The validator reads these. They are what a reviewer of the patch would be given.
- **The observations.** The validator reads what the tools returned during the run, as the runtime recorded it.
- **The messages.** The validator never reads this field, which holds the conversation.

The conversation is the proposer's account of the evidence, and it is kept from the validator on purpose. A judge in court weighs the evidence itself rather than a defendant's account of it. The validator is in the judge's position, so its verdict should rest on what the tools returned rather than on how the proposer described it. Otherwise a confident claim in the conversation, for example that the fix is complete, would reach the validator as if it were evidence.

Keeping the conversation away means that the validator does not read the proposer's account of its own work. This does not make the verdict correct, because the validator is still a model judging work.

4.12 Self-review: accuracy and cost

The validator calls a model to judge a patch, so its value depends on how reliable a model is when it judges work. Two studies measured this. The first shows that when a model reviews and revises its own answer without outside information, its accuracy can fall while its cost rises.

Intrinsic self-correction is the setting in which a model revises its answer with no external signal. An **external signal** is information from outside the model, such as a test result or the correct answer.

In one measurement, GPT-4 answered a benchmark of grade-school math word problems [2, Table 3]. Each round of self-review takes two further model calls. In the first, the model critiques its answer. In the second, it writes a revised answer. Table 10

compares no review with one and two rounds. Read the accuracy column against the column of model calls.

Table 10: Intrinsic self-correction by GPT-4 on grade-school math word problems: accuracy and the number of model calls with no review, one round of self-review and two rounds.

Who decides the answer is done	Accuracy	Model calls
Nobody: one call, no review	95.5%	1
The model reviews itself once	91.5%	3
The model reviews itself twice	89.0%	5

A single call with no review answered 95.5% of the problems correctly. One round of self-review lowered accuracy to 91.5% at three calls, and two rounds lowered it to 89.0% at five calls. Accuracy fell with each round, while the number of model calls rose to five times that of a single answer.

An external signal changes the result. The **ground-truth label** is the known correct answer. When the review stops as soon as the ground-truth label shows that the current answer is correct, so that a correct answer is never revised, accuracy rises instead. A deployed agent does not have this label at run time. If we knew the correct answer, we would not need the model to find it.

The measurement concerns one model on math problems, not program repair. It shows that, on this benchmark, a model's review of its own work without new information made the answers worse.

4.13 Model verifier and sound verifier

A **verifier** is the component that checks a proposed answer and accepts or rejects it. Our validator is a verifier, and so are the submit checks. A **model verifier** is a verifier that is itself a language model, for example the model that produced the answer. A **sound verifier**, in the sense used here, is a program that checks an answer exactly against the rules of the task, so its verdict is always right.

Review improves results when the verifier is sound, and it can make them worse when the verifier is the model that produced the answer. A second study compared verifiers on 100 instances in each of several reasoning and planning domains [5, Table 1]. In each run, the model proposes an answer and the verifier checks it. A rejected answer goes back to the model with the verifier's critique for another attempt. Only the verifier changes between conditions. The results below come from these domains:

- **Graph coloring.** The task is to give each vertex of a graph one of a fixed number of colors, the fewest the graph allows, so that no two connected vertices share a color.
- **Game of 24.** The task is to combine four given numbers with arithmetic operations to make 24.
- **Blocksworld.** The task is to plan a sequence of moves that stacks and unstacks blocks to reach a goal arrangement.

Table 11 gives the results for graph coloring and Game of 24. Compare each row with the first, which is the model's first attempt without review.

With the model that produced the answer as the verifier, the share of instances solved fell below that of a single attempt: from 16% to 2% on graph coloring and from 5% to 3% on Game of 24. With a sound verifier, the shares rose to 38% and 36%.

Blocksworld is the exception. There, review by the model helped, raising the result from 40% to 55%, and a sound verifier still did better. The exception shows that a model's critique is not always harmful. It does not change the finding that the largest gains came from a sound verifier.

Table 11: Percentage of 100 instances per domain solved correctly with no review, with review by the model that produced the answer, and with a sound verifier.

Who decides the answer is good enough	Graph coloring	Game of 24
Nobody: one attempt, no review	16%	5%
The model that produced it	2%	3%
A sound verifier	38%	36%

4.14 What the studies imply for a model as verifier

Both studies concern puzzles and math problems, not program repair, so they give no rate we can expect for our validator. They do support three conclusions about using a model to judge work:

- **A model's review of its own answer is not an external signal.** When the verifier was the model that produced the answer and it had no new information, review lowered accuracy in every comparison above except Blocksworld. In the study that counted model calls, review also raised the number of calls from one to as many as five.
- **The largest gains came from information the model did not have.** Results improved most when the procedure had the ground-truth label or a sound verifier. The one gain from self-review, on Blocksworld, was smaller than the gain from a sound verifier.
- **Our checks are not a sound verifier.** The rescan is a program, but it is not always right. An edit that only renames the flagged expression can clear the alert and leave the weakness in place.

These conclusions bear on the design of our validator. In the studies, review helped most when the verifier had information from outside the model that produced the answer. Our validator is a separate model call. It reads what the tools returned and the results of the checks rather than the proposer's account of them. It also sees only patches that have already passed the mechanical checks. Neither feature gives it a ground-truth label or a sound verifier, so neither makes its verdict always right. The validator is a second opinion from a model, placed after checks that are cheap and reproducible but not sound.

The planner and the validator are two of many agent architectures. Section 5 examines others, drawn from a catalogue of design patterns.

Key points

- The reactive loop records what it has seen but not what it intends to do, so it cannot measure its progress, and a plan written into the state records that intent.
- Open-loop planning changes nothing in response to the environment, implicit closed-loop planning changes only the next action, and explicit closed-loop planning can replace the rest of the plan.
- On the first stall, the runtime, never the model, writes `failed_step`, so the run goes to the planner for one revision, and a second stall ends the run with `no_progress`.
- Each plan step records a goal and the evidence that would show it is done, and the plan reaches the controller as a human message rather than as one of its own replies.
- In one study of action selection, scoring an action by what follows it made early dead ends rarer and recovery more likely, while wider search with the same myopic score made first-move dead ends more frequent.
- With a validator, the mechanical checks in `submit` become a

gate, and `validate`, which reads the evidence and never the conversation, is the only node that can accept a patch.
- In most measurements, a model reviewing its own answers without an external signal lowered accuracy at higher cost, and the largest gains came from sound verifiers, which our rescan is not.

5 Other agent architectures

The previous sections took `mini-swe-agent` through a fixed workflow, a reactive loop (ReAct), a planner and a validator. Each time, the agent, the alert and the tools stayed the same, so only the architecture changed. We now survey further architectures from a published catalogue of agentic design patterns. We sort them by who orchestrates them, that is, by who decides which component runs next. The answer determines whether we can read the order of a run from the program. It also shapes what a pattern costs and how easily we can find its mistakes.

5.1 Design patterns and who orchestrates them

An **agentic design pattern** is a reusable arrangement of model calls, agents and the logic that connects them, described independently of any particular task. A published catalogue describes twelve such patterns [1], and we study eleven of them.

Each pattern describes an agent architecture in the sense of the previous sections: it fixes which components exist and who decides what runs next. A model call is a single request to a model and its reply. In these patterns, an agent is a component built around a model. It may make several model calls, and often use tools, to finish its part of the work.

Several of the patterns use more than one agent, so we name the roles an agent can play inside a larger system:

- **Subagent.** A subagent is an agent that another agent, or the program that runs the agents, calls to carry out one part of the task.
- **Specialist.** A specialist is a subagent built for one kind of work.
- **Worker.** A worker is a subagent that carries out its part of the task directly, without dividing it further.

Orchestration is the work of deciding which component runs next, what input it receives, and when the task is finished. These are control decisions, the decisions that determine how a run proceeds (Section 2.1). The component that makes them is the orchestrator, the component that Unit 1 of the course studies (Course context).

We sort the patterns by who orchestrates them. Most patterns fall on one side of a single division: either a model orchestrates, or predefined logic does. The remaining patterns give other answers.

Predefined logic is code, written at design time, that arranges the components in a fixed structure. The structures in the catalogue are a sequence, a set of workers whose results are gathered, and a loop. Predefined logic fixes which components run, how they are arranged and what ends the work, as the steps of a fixed workflow are fixed (Section 2). When a model orchestrates, it makes these decisions at run time from what it has read, as the model in a reactive loop chooses each next action (Section 3).

The eleven patterns we study answer the question of who orchestrates as follows:

- **The model.** A model orchestrates the reason and act pattern, the coordinator pattern and the hierarchical task decomposition pattern. In reason and act, the model works inside a single agent. In hierarchical task decomposition, models orchestrate at several levels.
- **Predefined logic.** Predefined logic orchestrates the sequential, parallel and loop patterns. It also runs the review and critique pattern and the iterative refinement pattern as loops.
- **Predefined checkpoints.** In the human-in-the-loop pattern, the run stops at points fixed at design time and waits for a person to decide. A checkpoint here is such a pause for a person's decision. It differs from the checkpoint that saves a run's progress so that the run can resume (Section 2.5).
- **Code the team writes.** In the custom logic pattern, the team that builds the system writes its own conditional code, which combines the other patterns.
- **No central orchestrator.** In a swarm, no single component decides what runs next.

The answer matters because it determines which control decisions are known before a run starts. When predefined logic orchestrates, we can read the possible orders of execution from the program, as we could for a fixed workflow. When a model orchestrates, the order is chosen during the run. Only the run's trace, the log of events that the runtime writes during a run (Section 3.4), then shows what was chosen.

Some of the patterns restate architectures we have already studied. The reason and act pattern is the reactive loop of Section 3. The sequential pattern has the shape of a fixed workflow. The review and critique pattern separates the component that proposes a result from the component that accepts it, as the validator does (Section 4.10).

Figure 8 shows the first six patterns as the catalogue draws them. In each diagram, the user's prompt enters on the left, and the arrows show which components run and in what order. In the reason and act diagram, the memory box plays the role of the run's state. Each observation is saved there, and the next thought reads it.

In the sequential, parallel and loop diagrams, the box that receives the prompt is labeled an agent (sequential agent, parallel agent, loop agent). That box is the predefined logic. It runs its subagents in an arrangement written before the run, and no model decides the order. In the diagrams of the review and critique pattern and the iterative refinement pattern, the generator is labeled a loop agent. The label shows that the generator runs inside a loop that predefined logic controls.

5.2 Reason and act

In the **reason and act** pattern, one agent repeats a cycle of thought, action and observation (Figure 8a). The pattern is the reactive loop of Section 3, and the model decides every next step.

In the thought, the model reasons about what to do next. In the action, it calls a tool. In the observation, it reads the tool's result. The cycle repeats until the model gives an answer, the run reaches a cap on the number of iterations, or an error stops it. The cap plays the role of the budget, the limit on model calls that stops a reactive loop (Section 3.10).

- **Who orchestrates.** The model orchestrates, inside one agent. No other component chooses the next tool call. The model chooses it after reading the previous observation.
- **When to use it.** We use reason and act when the task is dynamic, meaning that the right next step depends on what the last step returned. In our running example, whether `mini-swe-agent` should read a helper function in another file depends on what its first read of the flagged code showed (Section 2.6).
- **When it is a poor choice.** Reason and act is a poor choice when a single query to the model would answer the user's request, the prompt that starts the run. Every cycle adds a model call, so the pattern's **latency**, the time the user waits for the response, is higher than that of a single call. Every observation also feeds the next decision, so a misleading tool result can carry through to the final answer.

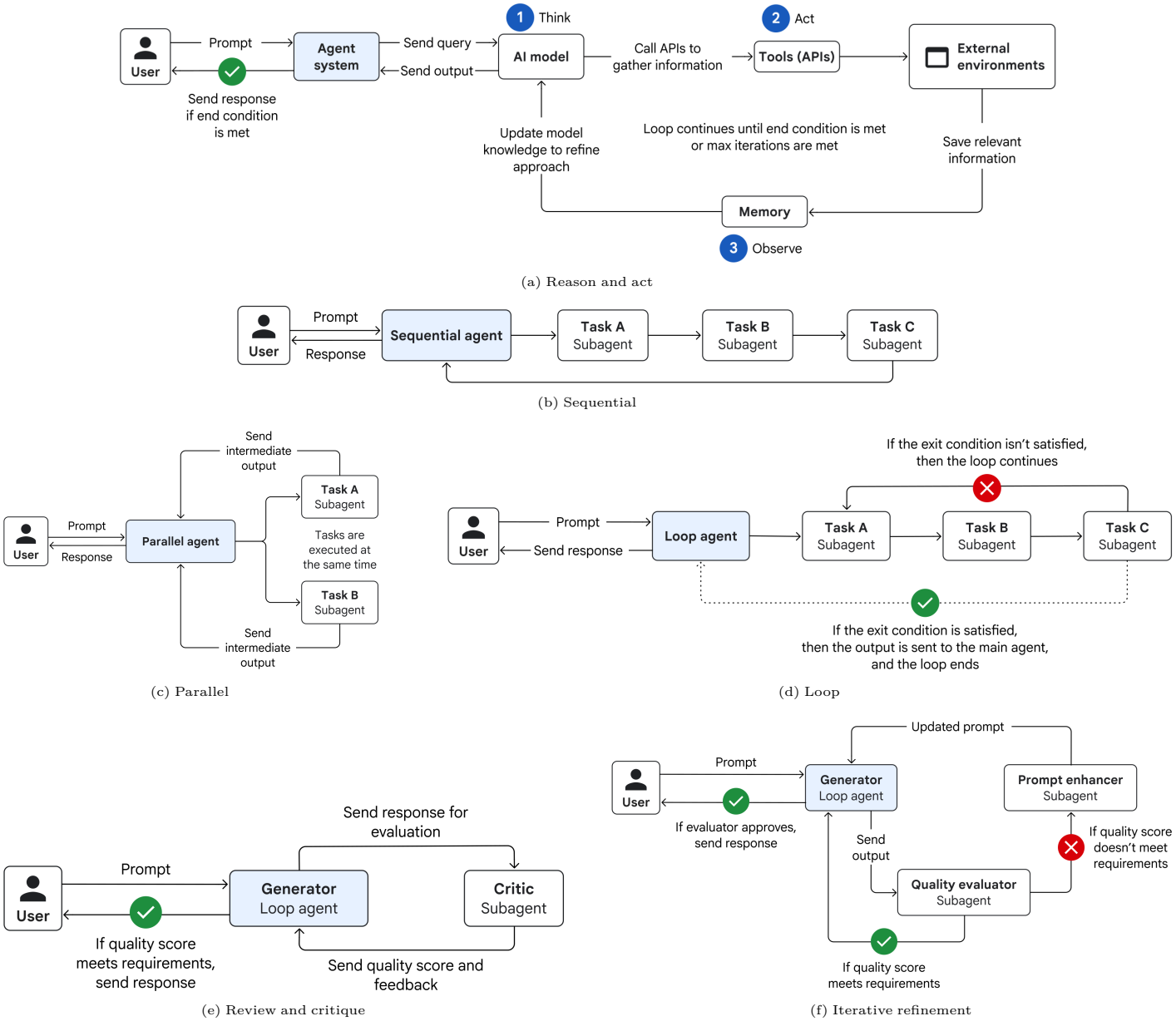


Figure 8: Patterns orchestrated by a model inside one agent (a) and by predefined logic (b to f). The review and critique pattern (e) and the iterative refinement pattern (f) are loops run by predefined logic. Google Cloud Architecture Center, agentic design patterns, CC BY 4.0.

5.3 Sequential, parallel and loop

In the sequential, parallel and loop patterns, predefined logic orchestrates. The program decides in advance which subagents run, how they are arranged, and what ends the work.

In the **sequential** pattern, specialists run one after another in a fixed linear order, and the output of each one is the input of the next (Figure 8b).

- **Who orchestrates.** Predefined logic orchestrates. The order is written into the program before the run, so the agents run in a fixed order, as the steps of a fixed workflow do.
- **When to use it.** The pattern suits a process in which every input needs the same steps in the same order, for example extracting data, then cleaning it, then loading it.
- **When it is a poor choice.** The sequential pattern is a poor choice when some runs do not need every step. The fixed order makes every run execute all of them, so a slow step that a run did not need still delays its response.

In the **parallel** pattern, independent workers carry out separate parts of the task at the same time (Figure 8c). A **gather step**, also called the join, then combines their results into one response.

- **Who orchestrates.** Predefined logic orchestrates. It decides which workers run and that the gather step receives all of their results.
- **When to use it.** We use the parallel pattern when the work splits into parts that do not depend on one another's results, for example collecting information from several sources at once.
- **When it is a poor choice.** The parallel pattern is a poor choice when the workers' results can conflict. The workers do not read one another's records, so nothing keeps their results consistent. The gather step must then decide which result to keep, or how to merge them, before it can produce one response. Every worker also makes its own model calls, so cost and token use grow with the number of workers.

In the **loop** pattern, a sequence of subagents runs repeatedly until an exit condition holds (Figure 8d). An **exit condition** is

a test that ends the loop when it is satisfied, such as a maximum number of iterations or a required state. It is usually checked after each pass, although the designer can place the check at any point in the sequence. The loop pattern differs from reason and act in who chooses the steps. In the loop pattern, the sequence of subagents is the same on every pass. In reason and act, the model chooses each next step.

- **Who orchestrates.** Predefined logic orchestrates, because both the sequence and the exit condition are written before the run.
- **When to use it.** We use the loop pattern when the work needs repeated passes, or needs to correct its own earlier output, before it is good enough.
- **When it is a poor choice.** The loop pattern is a poor choice when its exit condition is not clearly defined. A loop whose exit condition never holds keeps running until its budget, the limit on model calls, stops it. A budget guarantees that the loop stops. It does not guarantee that the work is finished (Section 2.4).

5.4 Loops of generation and evaluation

The review and critique pattern and the iterative refinement pattern are both loops run by predefined logic. In each of them, one component produces a result and another evaluates it.

In the **review and critique** pattern, a **generator** produces a **candidate**, an output proposed for acceptance, and a **critic** judges it against stated criteria (Figure 8e). The critic approves the candidate, rejects it, or returns a quality score with feedback, which the generator uses to revise. When a candidate meets the criteria, the generator returns it as the response. The generator proposes and the critic accepts, which is the separation that the validator makes for the patches of `mini-swe-agent` (Section 4.10).

- **Who orchestrates.** Predefined logic runs the pattern as a loop. The program sends every candidate to the critic and repeats the cycle until the critic approves.
- **When to use it.** We use review and critique when the output has to be accurate or has to meet strict rules, for example code that must pass an audit.
- **When it is a poor choice.** Review and critique is a poor choice when speed matters. Every evaluation is at least one more model call, and each revision the critic asks for adds a new candidate and a new evaluation.

In the **iterative refinement** pattern, one or more agents improve a result held in state over repeated cycles (Figure 8f). The cycles stop when the result reaches a **quality threshold**, a minimum score set in advance, or when the number of cycles reaches a cap. In the diagram, the generator's output goes to a quality evaluator. If the score meets the threshold, the generator returns the result. If it does not, a prompt enhancer, a subagent that rewrites the generator's prompt, produces a new prompt, and the generator tries again.

- **Who orchestrates.** Predefined logic runs the loop, as in review and critique.
- **When to use it.** We use iterative refinement when the task is hard to get right in one pass, for example drafting a document or debugging.
- **When it is a poor choice.** Iterative refinement is a poor choice when the exit condition is vague, for example a quality threshold that is never stated precisely. Each cycle costs time and money whether or not it improves the result.

The two patterns differ in what the loop is for. In review and critique, the loop decides whether to accept a candidate. In iterative refinement, the loop improves one result from cycle to cycle.

5.5 Coordinator and hierarchical task decomposition

In the coordinator pattern and the hierarchical task decomposition pattern, a model orchestrates other agents. It decides at run time how the task is divided and which agent does each part.

Figure 9 shows the coordinator, hierarchical task decomposition, swarm and human-in-the-loop patterns as the catalogue draws them. The catalogue draws custom logic only through one example, a refund process that runs a parallel check and then a conditional branch. We do not reproduce that example, because custom logic has no general shape: its shape is whatever conditional code the team writes.

In the **coordinator** pattern, a central agent reads the request, splits it into parts, and sends each part to a specialist (Figure 9a). The choice of which specialist receives each part is the coordinator's routing decision. The specialists send their results back to the coordinator, which assembles the response.

- **Who orchestrates.** A model orchestrates. The coordinator uses a model to make the routing decision, so the routing is decided at run time from the content of the request.
- **When to use it.** We use the coordinator pattern when the routing must adapt to the request. This is the case when we can tell which specialists a request needs only after reading it.
- **When it is a poor choice.** The coordinator pattern is a poor choice when a single agent could do the work, because the coordinator adds model calls, tokens and latency to every request.

In the **hierarchical task decomposition** pattern, a root agent splits the task into subtasks, and each subagent may split its subtask again (Figure 9b). Splitting continues until each piece is small enough for a worker to carry out directly. The result is a tree of agents, with the root at the top and the workers at the leaves.

- **Who orchestrates.** A model orchestrates at several levels, because every split is a model's decision. Every agent that splits its task plays the coordinator's role for the agents below it. In the diagram, the root agent is labeled a coordinator agent.
- **When to use it.** We use hierarchical task decomposition when the problem is open-ended and needs research, planning and synthesis together. A problem is open-ended when its subtasks cannot be listed in advance, and synthesis is the combining of findings into one answer.
- **When it is a poor choice.** Hierarchical task decomposition is a poor choice when the system has to be easy to debug. The tree has many agents and many model calls, so finding the decision that caused a wrong result is hard.

5.6 Swarm, human in the loop, and custom logic

The swarm, the human-in-the-loop pattern and custom logic each give a different answer to the question of who orchestrates. A swarm has no central orchestrator. The human-in-the-loop pattern is orchestrated by predefined checkpoints, and custom logic by code the team writes.

In a **swarm**, several specialists communicate with one another directly and refine a result together until an explicit exit condition is met (Figure 9c). Any specialist can pass the work to any other. A **handoff** is one such transfer of the work from one agent to another.

In the diagram, an entry agent labeled the dispatcher agent passes the request into the swarm. It returns the output once the exit condition is met. It does not direct the work inside the swarm. This agent is unrelated to the dispatcher that Lecture 4 studies, the runtime component that executes tool calls.

- **Who orchestrates.** No central component orchestrates. Each agent chooses for itself when to hand off and to whom, so no single component decides which agent acts next.

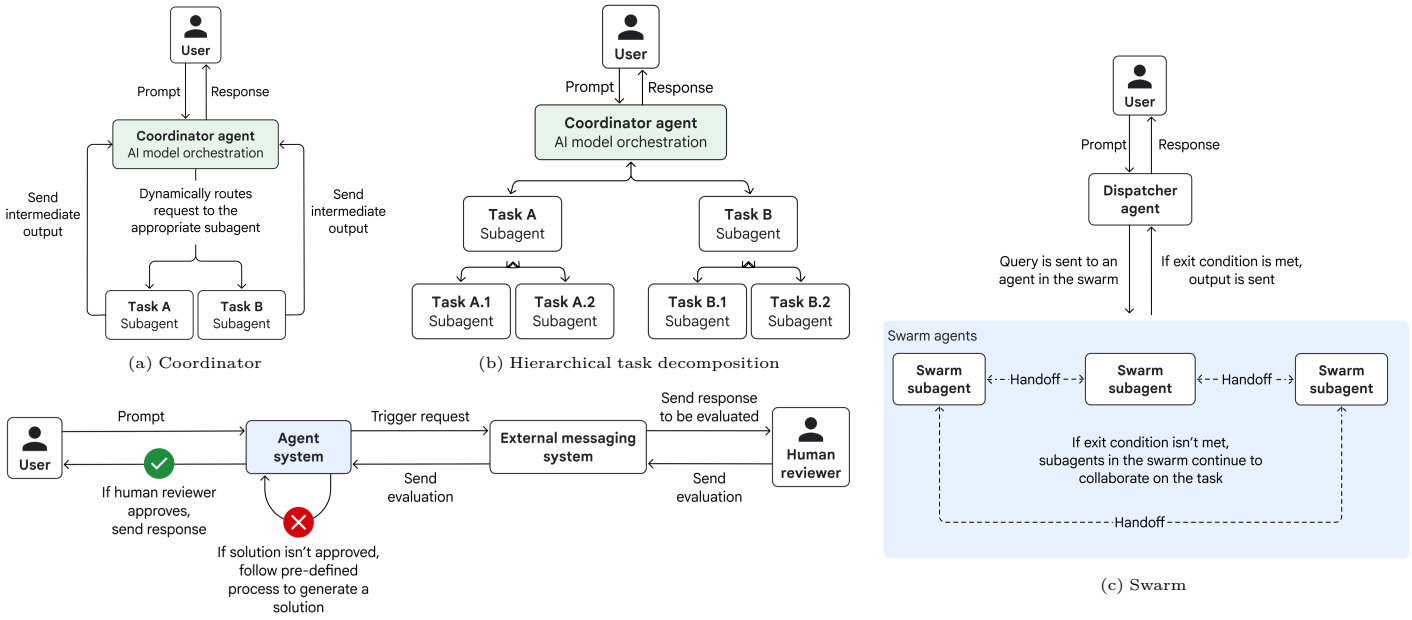


Figure 9: Patterns orchestrated by a model (a, b), with no central orchestrator (c), and by predefined checkpoints (d). Google Cloud Architecture Center, agentic design patterns, CC BY 4.0.

- **When to use it.** We use a swarm when the problem is ambiguous, that is, when it has no single agreed framing, and it benefits from argument between different perspectives.
- **When it is a poor choice.** In most other cases a swarm is a poor choice. Every handoff adds model calls, and the exchanges continue until the exit condition is met. The catalogue therefore describes the swarm as the most complex and costly multi-agent pattern to implement. A swarm may also never **converge**, which means that the agents may never reach a result that satisfies the exit condition.

In the **human-in-the-loop** pattern, the agent pauses at a checkpoint and waits for a person to approve its work, correct it, or supply missing input (Figure 9d). In the diagram, the agent sends its result through an external messaging system to a human reviewer. If the reviewer approves, the agent returns the response. If not, the agent follows a predefined process to produce a new solution.

- **Who orchestrates.** Predefined checkpoints orchestrate this pattern, because the points where the run stops for a person are fixed at design time.
- **When to use it.** We use the human-in-the-loop pattern when the decision is high-stakes or subjective, for example a payment or a sensitive document.
- **When it is a poor choice.** The human-in-the-loop pattern is a poor choice when no person is available to answer. The pattern also requires an interaction system, such as the messaging system in the diagram, that has to be built and then maintained.

In the **custom logic** pattern, the orchestration is conditional code that the team building the system writes, combining the other patterns as the task requires.

Custom logic is written before the run, as predefined logic is. The difference lies in the shape of the code. Predefined logic follows one of the fixed structures of the named patterns: a sequence, a set of workers whose results are gathered, or a loop. Custom logic can branch in any way the team chooses.

- **Who orchestrates.** Code that the team writes orchestrates. The team writes every branch and the condition that selects it, although a condition may test a value that the model produced.
- **When to use it.** We use custom logic when the branching goes beyond a linear sequence, or when fixed rules have to be mixed with model reasoning.
- **When it is a poor choice.** Custom logic is a poor choice when one of the named patterns already matches the task. The team would then own the whole flow and have to debug all of it.

5.7 Axes for comparing patterns

Several patterns can often perform the same task. Choosing among them requires comparing what each one costs and what each one lets us see. We make the comparison along **axes**, each a property of a pattern that can be examined across patterns. We apply the axes to patterns of both kinds. A model orchestrates the coordinator pattern and hierarchical task decomposition. Predefined logic orchestrates the parallel pattern and the review and critique pattern. Comparing patterns of both kinds shows how the choice between a model and predefined logic affects each axis.

Each axis asks one question about a pattern:

- **Task ownership.** Which component is responsible for the whole task, and which only for a piece of it?
- **State boundaries.** What can each component see, and what information has to cross from one component to another?
- **Model-call overhead.** How many model calls does the pattern's structure add, and how many calls does a request then cost in total?
- **Latency.** How much longer does the user wait for the response to a single request because of the pattern's structure? The answer depends on the **critical path**, the longest sequence of steps that must happen one after another before the response is ready, and on each **round trip**, the time taken to send work to a component and receive its result.
- **Failure propagation.** Where does a mistake go after it is made, and which component can notice it?
- **Inspectability.** What can a person reading the trace after the run reconstruct about which component decided what?

Table 12 summarizes the comparison. Each row is one axis, and reading a row across the columns shows how the patterns differ on that axis. The columns for the coordinator and for hierarchical task decomposition hold the patterns that a model orchestrates.

Table 12: The axes of comparison applied to patterns that a model orchestrates (coordinator, hierarchical task decomposition) and to patterns that predefined logic orchestrates (parallel, review and critique).

Axis	Coordinator	Hierarchical task decomposition	Parallel	Review and critique
Task ownership	The coordinator owns the task and delegates pieces. Each specialist owns only its piece.	Nested. Each level owns the decomposition it made.	Each worker owns one independent piece. The gather step owns the whole.	The generator owns the candidate. The critic owns acceptance.
State boundaries	Separate contexts. Only what the coordinator passes crosses.	One boundary per level. Partial results cross upward.	One state record per worker. The gather step reads all of them.	No new boundary. One shared record, with the critic's reasons added.
Model-call overhead	One routing call per request, plus the specialists' calls.	Multiplied by the depth of the hierarchy.	The sum of the workers' calls, made concurrently.	One critic call per candidate, plus one generator call per revision.
Latency	The routing call is on the critical path.	Each level adds a round trip.	Reduced to the slowest worker plus the gather step.	Each revision adds a round trip.
Failure propagation	A misrouted task fails inside the wrong specialist, and the coordinator may not notice.	A wrong decomposition spreads downward, and its cost multiplies.	Contained in each worker until the gather step, where a missing piece blocks the whole.	A wrong acceptance passes the candidate on. A wrong rejection costs a revision.
Inspectability	The routing decision and its reasons are in the trace, if recorded.	Traces nest. The trace must record which level made each decision.	Traces interleave. The trace must record which worker wrote each event.	Better than when the generator accepts its own output. Each acceptance carries the critic's reasons.

Comparing them with the other two columns shows where a control decision made at run time adds cost or risk.

5.8 Task ownership and state boundaries

Task ownership and state boundaries describe how a pattern divides responsibility and information among its components. Task ownership says which component is responsible for each part of the result. State boundaries say what each component can see.

Task ownership differs across the patterns as follows:

- **Coordinator.** The coordinator owns the whole task and delegates pieces of it. Each specialist owns only the piece it received.
- **Hierarchical task decomposition.** Ownership is nested. Each level owns the decomposition it made, and the workers at the leaves own only the pieces they carry out.
- **Parallel.** Each worker owns one independent piece. The gather step owns the whole, because it is the only component that receives every result.
- **Review and critique.** The generator owns the candidate, and the critic owns the decision to accept it.

A run's state is the record of the run so far, which its components read and write, and a model's context is the input it receives in one call (Section 3.1). A **state boundary** determines which parts of the state reach which model's context. It differs across the patterns as follows:

- **Coordinator.** The coordinator's context and each specialist's context are separate. Only what the coordinator passes to a specialist crosses, so a specialist works from the coordinator's description of its piece.
- **Hierarchical task decomposition.** There is a state boundary at every level. Partial results cross upward, from the workers to the agent that split their task, and from there toward the root.
- **Parallel.** Each worker keeps its own state record, and the gather step reads all of them. The workers do not read one another's records, which keeps them independent.
- **Review and critique.** The pattern adds no state boundary. The generator and the critic work on one record, as a single agent would, and the critic adds its reasons to it.

5.9 Model-call overhead and latency

Model-call overhead and latency measure what a pattern costs. Model-call overhead counts the model calls that the pattern's structure adds, such as the coordinator's routing call, and the total that a request then costs. Latency measures how much longer the user waits for the response to a single request.

A request can be no faster than its critical path. The patterns compare as follows:

- **Coordinator.** The overhead is one routing call per request, plus the specialists' calls. The routing call is on the critical path, because no specialist can start until the coordinator has decided where the request goes.
- **Hierarchical task decomposition.** The overhead is multiplied by the depth of the hierarchy, its number of levels, because every level that splits a task makes calls of its own. Each level also adds a round trip, because a level waits for the results of the level below it.
- **Parallel.** The overhead is the sum of the workers' calls, which are made concurrently. The latency falls to that of the slowest worker plus the gather step, because the workers run at the same time.
- **Review and critique.** The overhead is one critic call for each candidate, plus one generator call for each revision the critic asks for. Each revision also adds a round trip, because a new candidate has to be generated and then evaluated.

The parallel pattern shows that cost and latency are separate axes. Running the workers at the same time does not reduce the number of model calls, so the pattern pays for every worker. It waits only for the slowest worker, so it answers sooner than the same work done in sequence.

5.10 Failure propagation and inspectability

Failure propagation and inspectability concern mistakes. Failure propagation describes where a mistake travels after it is made. Inspectability describes what a person reading the trace afterwards can reconstruct about which component decided what, which is what finding a mistake requires.

Failure propagation differs across the patterns as follows:

- **Coordinator.** A misrouted task fails inside the wrong specialist. The coordinator sees only what the specialist returns, so it may not notice the misrouting.
- **Hierarchical task decomposition.** A wrong decomposition propagates downward. Every subtask below the wrong split inherits the mistake, so its cost multiplies with the number of subtasks.
- **Parallel.** A mistake stays inside the worker that made it, because the workers do not read one another's records. It reaches the rest of the system only at the gather step. There, a worker that returns no result leaves a piece missing, and the gather step cannot assemble the whole response.
- **Review and critique.** The critic can be wrong in either direction. A critic that accepts a wrong candidate passes it on as the output. A critic that rejects a correct one costs an unnecessary revision.

Inspectability differs as follows:

- **Coordinator.** The routing decision and its reasons are in the trace, provided we recorded them.
- **Hierarchical task decomposition.** Traces nest, because each level's trace contains the traces of the levels below it. The trace must therefore record which level made each decision.
- **Parallel.** Traces interleave: events from different workers are mixed together in the order they occur, because the workers run at the same time. The trace must therefore record which worker wrote each event.
- **Review and critique.** Inspectability is better than when the generator accepts its own output, because each acceptance carries the critic's reasons.

When a model orchestrates, inspectability takes the place of the address guarantee of a fixed workflow. In a fixed workflow, the address guarantee lets us find the step that produced a wrong result, because every step is known before the run (Section 2.4). When a model makes the control decisions, that guarantee no longer follows from the program, so the trace has to record who decided what.

The two kinds of orchestration also fail in different places. In the coordinator pattern and in hierarchical task decomposition, a model makes control decisions. A wrong routing or a wrong split is then a failure in its own right, and it shows up only later, in the components that receive the work. In the parallel pattern and in the review and critique pattern, the split and the loop are written before the run. Mistakes made during a run therefore arise in the workers' results and in the critic's judgment.

Key points

- An agentic design pattern is a reusable arrangement of model calls, agents and the logic that connects them, described independently of any particular task.
- We sort the patterns by who orchestrates them, that is, who decides which component runs next: a model (the reason and act pattern, the coordinator pattern and hierarchical task decomposition), predefined logic (the sequential, parallel and loop patterns, the review and critique pattern and the iterative refinement pattern), predefined checkpoints (the human-in-the-loop pattern), code the team writes (custom logic), or no central orchestrator (a swarm).
- When predefined logic orchestrates, we can read the possible orders of execution from the program before the run, whereas when a model orchestrates, only the trace shows which order was chosen.
- The review and critique pattern separates proposing a result from accepting it, as the validator does, and each evaluation costs at least one more model call.
- We compare patterns on task ownership (who is responsible for what), state boundaries (what each component can see), model-call overhead (the model calls a pattern's structure adds), latency (how much longer the user waits), failure propagation (where a mistake travels) and inspectability (what the trace lets us reconstruct about who decided what).
- Model-call overhead and latency are separate axes, because the parallel pattern pays for every worker's model calls but waits only for the slowest worker and the gather step.
- When a model makes the control decisions, the address guarantee of a fixed workflow no longer follows from the program, so the trace has to record which component decided what.

References

[1] Google Cloud Architecture Center. Choose a design pattern for your agentic AI system. https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system, 2026. Page last updated 2026-05-28; accessed 2026-09-16. Content licensed CC BY 4.0.

[2] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. An empirical study on failures in automated issue solving, 2025. arXiv:2509.13941v1, 17 September 2025.

[4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education Limited, Harlow, 4th, global edition, 2022. Global Edition of the fourth edition (Pearson, 2021). Quotations, figures, and page numbers in these notes are from this printing, in which Chapter 2, Intelligent Agents, occupies pp. 54–79. Section and figure numbers agree with the United States printing.

[5] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations (ICLR)*, 2025.

[6] Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. AdaPlanner: Adaptive planning from feedback with language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. Figure and table numbers follow the NeurIPS proceedings PDF.

[7] Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents, 2026. arXiv:2601.22311v1, 29 January 2026; section, table and figure numbers follow v1.

[8] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754. Article FSE037. Section and figure numbers in these notes follow the FSE camera-ready, not the arXiv preprint (arXiv:2407.01489), which carries the "Agentless" title prefix and a different section numbering.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. ICLR version, arXiv:2210.03629v3.